언리얼 엔진 5 렌더링 원리와 최적화

게임 배경 아티스트를 위한 렌더링 기술 포스트 모음

배경의 외관은 모델과 텍스처만으로 정해지지 않습니다. 표면의 정보가 빛과 만나고 화면으로 합쳐지는 과정에 따라 같은 에셋도 다르게 보입니다. 이 글들은 그 과정에서 각 기술이 하는 일과, 그것이 제작의 선택에 어떤 의미를 가지는지 풀어냅니다.

렌더링 파이프라인에서 시작해 물질과 반사, 지오메트리와 텍스처, 조명과 시간적 화질, 성능 분석으로 이어지는 26편의 포스트를 묶었습니다. 각 글은 독립적으로 읽을 수 있으며, 개념을 설명하는 본문과 이를 보여 주는 이미지·제작 사례로 구성했습니다.

이미지와 자료에 대하여

직접 제작한 개념도와 Unreal Engine 캡처, 외부 참고 이미지를 구분해 표시했습니다. 외부 이미지에는 원저자와 출처, 버전·편집 정보를 붙였습니다. 개념도와 진단 화면은 설명 자료이며 해당 프로젝트의 실행 시간을 입증하는 자료가 아닙니다.

내용은 UE 5.8 계열 데스크톱 경로를 중심으로 설명합니다. 과거 버전의 외부 자료는 캡션에서 구분하며, 기능 지원과 실제 비용은 프로젝트의 플랫폼·설정에 따라 달라질 수 있습니다.

목차

렌더링 기초

01 렌더링 파이프라인 · 3
02 Forward와 Deferred · 8
03 GBuffer와 표면 정보 · 10
04 DBuffer와 데칼 · 12
05 Depth와 Early Z · 14
06 Overdraw와 화면 겹침 · 16

머티리얼과 빛

07 PBR과 재질 입력 · 18
08 BRDF와 반사 분포 · 20
09 Fresnel과 시선 각도 · 22
10 셰이더와 머티리얼 · 24
11 Substrate와 물질 구성 · 26
12 Translucency와 합성 · 28
13 SSS와 표면 아래 산란 · 30

지오메트리와 텍스처

14 Draw Call과 제출 작업 · 32
15 Instancing과 반복 배치 · 34
16 LOD와 화면 크기 · 36
17 Nanite와 가상화 지오메트리 · 38
18 Culling과 가시성 · 40
19 Mipmap과 텍스처 샘플링 · 42
20 Texture Streaming · 44

라이팅과 화면 품질

21 Lumen과 간접광 · 46
22 Virtual Shadow Maps · 48
23 TAA와 프레임 히스토리 · 50
24 TSR과 시간적 재구성 · 52

성능 분석

25 CPU와 GPU 병목 · 54
26 GPU Profiler와 개선 검증 · 56

용어 찾아보기 · 58

기술 메모 · 64

출처 목록 · 65

01 렌더링 파이프라인 장면이 이미지가 되기까지

배경 작업은 모델과 텍스처를 만드는 데서 끝나지 않습니다. 같은 에셋도 조명과 카메라, 렌더링 설정에 따라 전혀 다른 화면이 됩니다. 그 사이에서 무슨 일이 일어나는지 이해하는 출발점이 렌더링 파이프라인입니다.

렌더링 파이프라인은 모델·재질·조명·카메라로 이루어진 장면 데이터를 계산해, 화면에 표시할 픽셀의 색으로 만드는 일련의 처리 과정입니다. 파이프라인이라는 말은 한 작업의 결과를 다음 작업의 입력으로 넘기는 구조를 뜻합니다.

3D 장면은 그 자체로 사진이 아닙니다. 물체가 어디에 있고, 화면의 어디를 덮으며, 어떤 표면에 어떤 빛이 도달하는지 계산해야 이미지가 됩니다. 파이프라인은 이 일을 여러 단계로 나누고 필요한 중간 정보를 전달하는 렌더링의 전체 구조입니다.

장면을 여러 계산으로 나누는 이유

입력은 메시의 정점·배치, 텍스처와 재질, 광원, 카메라, 렌더링 설정입니다. 출력은 표시할 2D 이미지입니다. 중간에는 화면 위치, 깊이, 표면 특성, 조명 결과 같은 정보가 생깁니다.

처리 단계	입력으로 받는 정보	다음에 전달하는 결과
그릴 작업 준비	메시·배치·재질·카메라·장면 상태	처리할 후보와 그리기 명령
형태를 화면에 대응	정점 위치와 변환·카메라	화면의 삼각형·샘플과 보간 속성
깊이와 표면 계산	표면 후보·텍스처·재질	앞뒤 관계와 색·거칠기·법선 등
조명과 그림자 반영	표면 속성·광원·가림과 간접광	조명된 표면의 색
합성과 후처리	표면 색·반투명·노출과 효과	표시할 최종 이미지

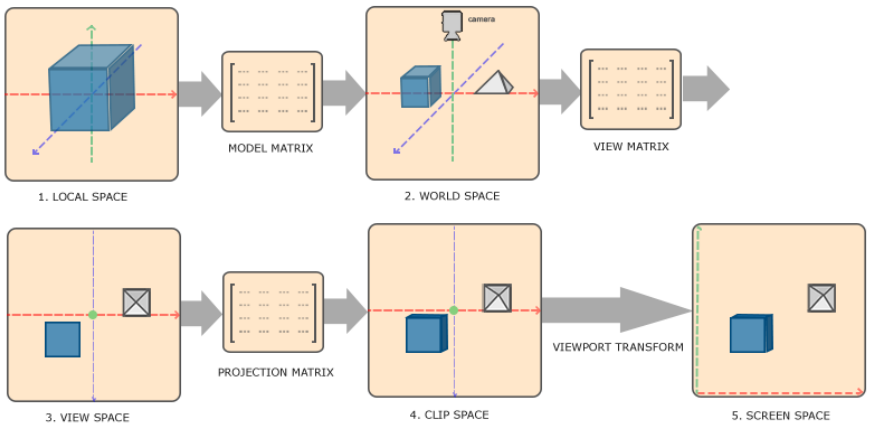
대표적인 래스터 렌더링의 데이터 의존 관계입니다. 그림자 등의 별도 갈래와 병렬 실행을 생략한 설명이며 고정된 GPU 실행 순서표는 아닙니다.

Epic 공식 구조 설명 [62]

형태를 화면에 옮기는 동안

그릴 작업을 준비합니다. CPU는 장면 상태와 카메라를 바탕으로 후보를 정리하고 명령을 준비합니다. 렌더 스레드는 이 준비를 맡는 CPU 실행 흐름이며, RHI는 명령을 DirectX·Vulkan 같은 그래픽 API에 연결하는 계층입니다. GPU는 제출된 일을 실행합니다.

메시는 삼각형으로 표면을 표현하고, 정점은 그 꼭짓점입니다. 정점 셰이더는 모델 내부의 위치에 장면 배치와 카메라 변환을 반영합니다. 같은 물체도 카메라가 달라지면 화면의 다른 위치와 크기로 보이는 이유입니다.



외부 참고 그림 · 모델의 로컬 공간부터 월드·카메라·클립·화면 공간까지 이어지는 좌표 변환입니다.

1→2에서는 큐브를 장면에 배치하고, 2→3에서는 카메라를 기준으로 장면을 해석합니다. 4→5에서는 투영한 결과가 화면 영역에 대응합니다. UE와 OpenGL의 축·깊이 규약은 같지 않지만 좌표 기준을 바꾸는 목적은 같습니다.

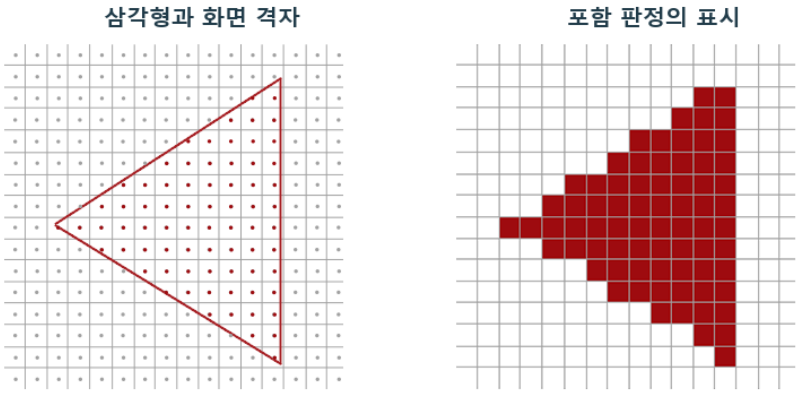
© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [37]
원본 전체 유지 · 비율 유지 · CC BY 4.0
이용 조건

모델 내부의 좌표는 장면의 좌표와 다릅니다. 회전과 확대가 없는 경우, 원점을 월드 X=10에 놓은 모델의 로컬 X=1인 점은 월드 X=11이 됩니다. 이후 카메라 기준 표현과 투영을 거쳐 화면 위치가 됩니다.

위치가 화면에 대응되는 것과 표면의 무늬가 정해지는 것은 별개의 일입니다. UV는 텍스처에서 읽을 위치를, 법선은 표면이 향하는 방향을 전달합니다. 위치·무늬·빛 반응의 방향은 서로 연결되지만 같은 데이터는 아닙니다.

삼각형 내부에도 값이 필요한 이유

화면으로 옮긴 삼각형은 아직 픽셀 색이 아닙니다. 래스터화는 삼각형이 화면의 어느 샘플 위치를 덮는지 판정하는 과정입니다. 샘플은 검사를 수행하는 화면 위치이며, 가려짐 등의 검사를 거쳐 실제 화면에 기여할 후보가 남습니다.



외부 비교 그림 · 삼각형과 화면 격자, 그리고 포함 판정을 표시한 결과를 나란히 배치했습니다.

왼쪽의 매끈한 삼각형 경계와 오른쪽의 격자에 맞춘 계단형 경계를 비교합니다. 이 그림은 래스터화와 앨리어싱의 관계를 설명하며 TAA 또는 TSR의 전후 결과가 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [38]
원본 전체 유지 · 비율 유지 · CC BY 4.0
이용 조건

삼각형의 꼭짓점에만 UV가 있어도 가운데에는 텍스처가 이어져 보입니다. 보간으로 알려진 꼭짓점 값 사이의 중간값을 구하기 때문입니다. 원근 투영에서는 이에 맞는 보정이 적용돼, 표면 내부에서 필요한 값을 연속적으로 얻을 수 있습니다.

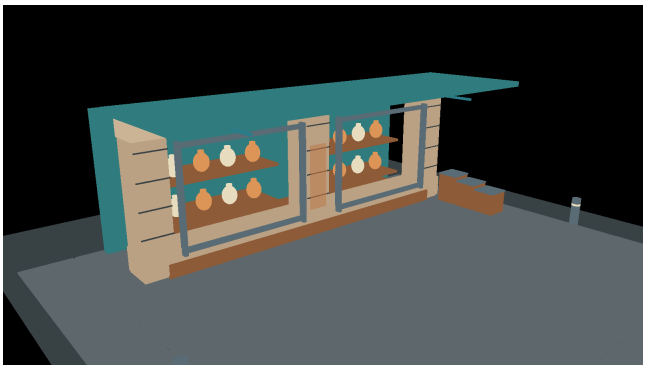
이 관계는 텍스처의 세밀함과 실루엣의 세밀함을 구분해 줍니다. 큰 삼각형 안의 무늬는 세밀할 수 있지만 바깥 윤곽이 둥글게 바뀌지는 않습니다. 표면이 흐린 문제와 형상이 각진 문제를 같은 수정으로 해결하기 어려운 이유입니다.

표면 정보가 빛을 만나 최종 색이 된다

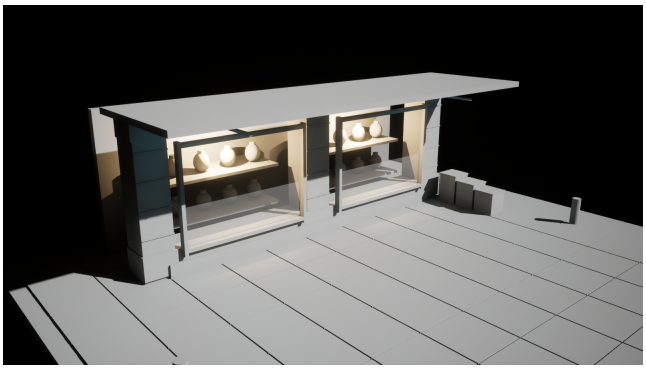
재질은 해당 표면의 기본 색과 거칠기, 법선 같은 정보를 계산합니다. 일반적인 Deferred 경로에서는 Base Pass가 이를 GBuffer에 기록하고, 조명 작업이 그 정보를 읽습니다. 버퍼는 계산 중 데이터를 보관하는 메모리이고 패스는 특정 목적의 작업 단위입니다.

기본 색은 물체의 최종 사진색이 아닙니다. 광원의 방향과 밝기, 주변의 간접광과 반사, 그림자가 더해져 조명된 색이 됩니다. 그림자는 광원 관점의 가림 정보도 필요하므로 같은 메시가 카메라 표면과 별도 작업에 참여할 수 있습니다.

Base Color



Lighting Only



직접 제작·동일 카메라의 실제 엔진 캡처. 표면색과 조명 기여를 분리해 보여 주는 진단이며, 두 화면의 단순 합이 Lit가 되는 분해식은 아닙니다.

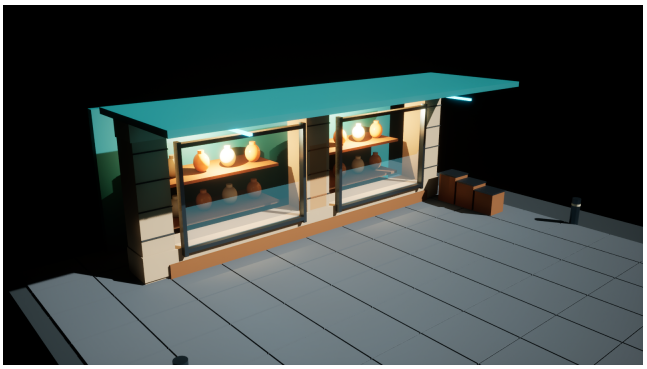
유리와 연기처럼 뒤가 비치는 표면은 배경의 색과 합성돼야 합니다. 이후 노출과 톤매핑, Bloom 등의 처리가 표시할 이미지를 구성합니다. 톤매핑은 넓은 장면 밝기를 표시 범위로 옮기고 Bloom은 밝은 부분을 주변으로 퍼뜨립니다. 효과의 정확한 적용 위치는 렌더 경로에 따릅니다.

아티스트의 작업은 어느 계산을 바꿀까

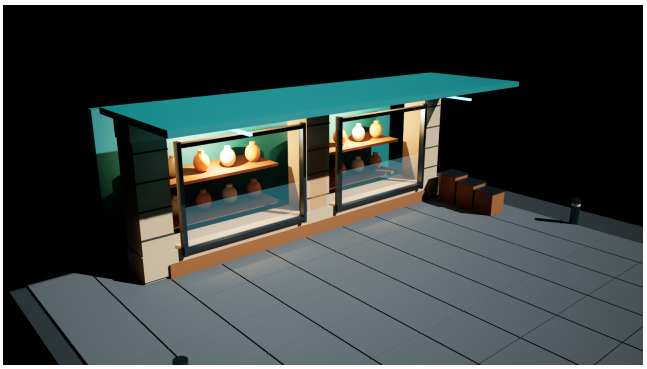
윤곽과 굽힘은 정점·삼각형·변형의 문제입니다. 큰 곡선의 실루엣은 텍스처 해상도만 올려 해결되지 않습니다. 반면 표면 무늬와 재료의 미세한 인상은 텍스처·UV·색·거칠기·법선과 연결됩니다. 흐린 무늬와 각진 형상은 서로 다른 수정 대상을 가집니다.

표면 입력이 같아도 광원과 그림자, 간접광과 환경 반사가 달라지면 외관은 바뀝니다. 최종 번짐과 색에는 발광·노출·톤매핑·Bloom도 관여합니다. 밝은 번짐이 과한 것을 기본 표면색만 낮춰 고치면 다른 조명에서 재료 자체가 어두워질 수 있습니다.

Bloom 적용



Bloom 해제



직접 제작·동일 구도의 실제 엔진 비교. 기본 장면과 Bloom 해제 상태이며 캡처 중의 시간은 성능 실측으로 사용하지 않습니다.

쇼윈도 캡처에서 Bloom을 끈 쪽은 밝은 바 주변의 번짐이 줄어듭니다. 이것은 벽의 기본 색이나 메시 형상을 바꾼 결과가 아니라, 밝은 화면 정보를 처리하는 후반부의 차이입니다. 눈에 보이는 밝음을 모두 텍스처의 밝기로 취급할 수 없는 이유가 여기에 있습니다.

폴리곤을 줄이는 일과 픽셀 일을 줄이는 일

한 메시도 깊이·카메라 표면·그림자 등 여러 작업에 참여할 수 있습니다. 삼각형 수, 그리는 화면 면적, 재질 계산, 광원과 중간 데이터의 읽기·쓰기가 서로 다른 비용을 만듭니다. 무엇을 줄일지는 실제로 느린 작업을 확인한 뒤 정합니다.

렌더링 파이프라인을 안다는 것은 패스 이름을 순서대로 외우는 일이 아닙니다. 내가 바꾼 데이터가 어떤 계산을 거쳐 화면에 나타나는지 이해하는 일입니다. 그 연결이 보이면 더 세밀하게 만들 대상과 단순하게 줄일 대상을 구분할 수 있습니다.

참고 자료

[01] Rendering Settings

[02] Render Dependency Graph

[03] Introduction to Performance Profiling and Configuration

[62] Graphics Programming Overview · Epic

02 Forward와 Deferred 조명 계산의 차이

렌더러 선택은 프로그래머만의 설정처럼 보이지만, 사용할 재질과 반사, 조명과 안티앨리어싱의 조건을 함께 정합니다. Forward와 Deferred의 차이도 어느 이름이 더 최신인지보다 표면과 빛을 어디에서 결합하는지로 이해하는 편이 정확합니다.

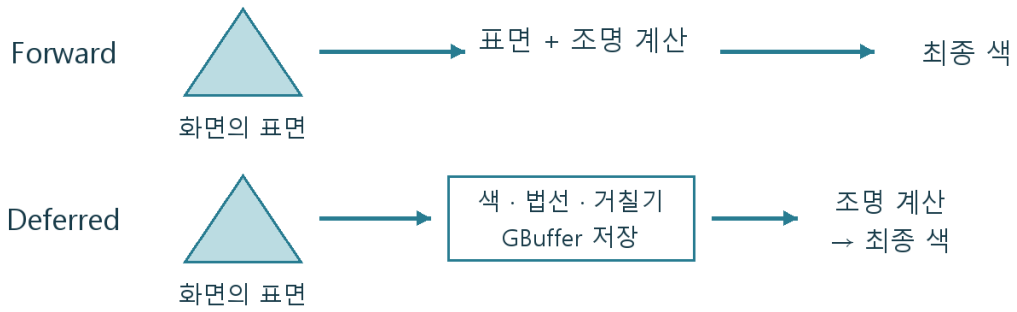
Forward와 Deferred는 표면의 재질 정보와 조명을 결합하는 방식을 나누는 대표적인 렌더링 구조입니다. 핵심 차이는 표면을 그릴 때 조명까지 평가하는지, 표면 정보를 먼저 저장하고 뒤에서 조명을 평가하는지입니다.

둘 다 장면을 조명된 이미지로 만들지만 작업을 나누는 위치가 다릅니다. 그래서 필요한 중간 버퍼, 기능 지원과 비용의 분포도 달라집니다. 프로젝트의 렌더러 선택은 재질·조명·AA 제작 조건을 정하는 상위 결정입니다.

바로 빛을 계산할까 표면부터 저장할까

두 경로가 출발하는 장면 데이터는 같습니다. 보이는 형상과 재질, 광원이 있어야 합니다. 차이는 이 정보로 조명된 색을 바로 만드는지, 조명이 읽을 표면 데이터를 먼저 만드는지에 있습니다.

Forward에서는 표면을 그리는 셰이더가 영향을 주는 광원과 재질 반응을 함께 계산합니다. UE는 광원을 선별하는 구조를 사용하므로 모든 표면이 모든 광원을 무조건 계산하는 방식은 아닙니다. Deferred에서는 먼저 보이는 표면의 색·법선·거칠기 등을 기록합니다. 다음 조명 작업이 이 값을 읽어 해당 화면 위치의 빛 반응을 계산합니다. 표면 정보와 조명 평가가 분리됩니다. 이후 반투명과 후처리를 포함한 화면 구성이 이어집니다. 선택한 렌더러에 따라 쓸 수 있는 기능과 구현 경로가 달라지므로, 같은 기능이 켜졌다는 이름만으로 동일한 결과라고 가정하지 않습니다.



차이의 중심은 조명 전에 표면 데이터를 따로 저장하고 읽는가입니다.

Forward는 표면을 그릴 때 빛을 평가하고, Deferred는 표면 데이터를 저장한 뒤 빛을 계산합니다. 이 분리 방식이 중간 버퍼와 기능 조건의 차이로 이어집니다.

렌더러 선택이 제작 조건을 바꾸는 이유

VR의 경계 품질, 필요한 반사·조명 기능, 플랫폼의 제약을 먼저 정할 때 사용합니다. 에셋을 만든 뒤 빠르다는 말만 듣고 렌더러를 바꾸면 기존 재질과 조명 설계를 다시 검토해야 할 수 있습니다.

여러 라이트가 겹치는 실내와 경계 안정성이 중요한 VR은 우선순위가 다릅니다. 그렇다고 전자는 무조건 Deferred, 후자는 무조건 Forward라는 공식이 생기는 것은 아닙니다. 필요한 표현을 어떤 경로에서 어떤 비용으로 제공하는지가 선택의 근거입니다.

빠른 렌더러라는 고정 답이 없는 이유

Deferred는 화면 크기의 중간 표면 데이터를 쓰고 읽는 부담이 있습니다. Forward는 표면 작업에 조명 계산이 포함됩니다. 광원·그림자·재질·해상도와 기능 조건에 따라 총 비용을 비교해야 합니다.

Forward와 Deferred는 같은 목표를 다른 작업 구조로 달성합니다. 기능이 빠진 결과의 빠른 숫자와, 필요한 화면을 유지한 결과의 효율은 구분해야 합니다. 렌더러 선택은 그 차이를 포함한 제작 조건의 선택입니다.

참고 자료

[01] Rendering Settings

[04] Forward Shading Renderer

03 GBuffer 최종 색 뒤에 있는 표면 정보

벽이 지나치게 번들거리면 라이트를 약하게 만들고 싶어집니다. 하지만 거칠기 입력이 잘못됐다면 조명으로 보정한 결과는 다른 환경에서 다시 무너집니다. 최종 화면 뒤의 재질 정보를 따로 볼 수 있게 해 주는 것이 GBuffer입니다.

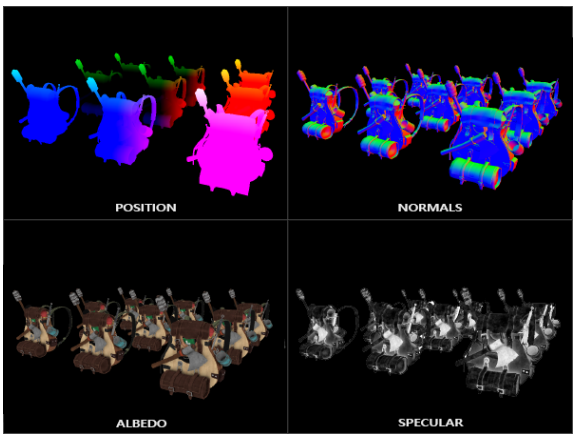
GBuffer는 Deferred 조명이 사용할 표면 특성을 화면 위치별로 저장하는 버퍼 묶음입니다. 버퍼는 계산에 쓸 데이터를 보관하는 메모리이며, GBuffer는 최종 사진이 아닌 표면 정보의 중간 저장소입니다.

표면의 기본 색과 방향·거칠기를 조명 계산과 분리합니다. 이 데이터를 기록한 뒤 여러 조명과 후속 계산이 재사용할 수 있습니다. 재질 입력과 조명 결과를 따로 진단할 수 있는 이유도 여기에 있습니다.

빛을 계산하기 전에 남겨 두는 데이터

텍스처와 재질이 만든 표면값은 해당 화면 위치의 데이터로 남습니다. 조명은 깊이와 카메라, 광원 정보와 함께 이 값을 읽어 최종 표면색을 계산합니다.

표면을 그리는 Base Pass에서 텍스처와 재질 입력을 평가합니다. Base Color는 기본 표면색, Roughness는 반사 분포의 거칠기, 법선은 표면 방향을 나타냅니다. 계산한 속성을 정해진 형식으로 저장합니다. Scene Depth는 별도의 깊이 정보입니다. 모든 속성이 각각 독립된 이미지 하나에 저장되는 것은 아니며, 채널 배치는 렌더러와 저장 형식에 따라 달라집니다. 조명 단계가 저장한 값을 읽고 빛의 방향·세기와 표면 반응을 결합합니다. 같은 표면 데이터도 조명이 바뀌면 최종 색이 달라집니다. Buffer Visualization은 일부 정보를 관찰용으로 표시합니다.



최종 조명색 대신 표면 속성을 따로 표시한 외부 GBuffer 사례입니다. 진단 화면의 색 배치는 실제 저장 채널 배열을 그대로 보여 주는 것은 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [39]
원본 전체와 비율 유지 · CC BY 4.0
이용 조건

반짝이는 벽을 조명만으로 고치기 어려운 이유

벽의 번들거림은 Roughness, 반사의 방향 오류는 World Normal, 고정된 색 얼룩은 Base Color부터 확인할 수 있습니다. 진단이 의도와 다르면 재질 입력을, 입력이 맞으면 조명과 반사 환경을 조사합니다.

같은 벽의 거칠기가 낮아지면 반사가 좁고 선명하게 보일 수 있습니다. 이때 Base Color가 같아도 Lit의 인상은 달라집니다. 반대로 광원만 움직이면 표면의 기본 값이 그대로인 상태에서 밝은 부분의 위치가 바뀝니다. 두 변화는 최종 색만으로는 쉽게 뒤섞입니다.

중간 정보를 저장하는 데도 비용이 든다

저장할 화면 픽셀 수와 픽셀당 데이터가 메모리 사용과 대역폭을 좌우합니다. 머티리얼 노드 수가 많아졌다는 사실만으로 GBuffer의 바이트 수가 같은 비율로 늘지는 않습니다.

GBuffer는 최종 화면을 한 번 더 찍은 사진이 아니라 조명이 읽는 표면 데이터입니다. 이 차이를 이해하면 재질을 고칠 문제와 빛을 고칠 문제를 같은 밝기 조절로 처리하지 않게 됩니다.

참고 자료

[02] Render Dependency Graph

[05] Programming with Substrate GBuffer Formats

04 DBuffer 데칼은 벽의 무엇을 바꾸는가

같은 벽 재질을 반복해도 빗물 자국과 손상 데칼이 더해지면 공간의 인상이 달라집니다. 데칼이 자연스럽게 보이는 이유는 단순히 얼룩 그림을 화면 위에 붙여서가 아닙니다. 빛을 받을 표면 정보를 바꾸는 과정이 있기 때문입니다.

DBuffer는 데칼이 바꾸는 표면의 색·법선·거칠기 정보를 기본 표면 계산 전에 준비하는 버퍼입니다. 데칼은 오염·손상·표식을 다른 표면에 투영하는 표현이고, DBuffer는 그 변경을 전달하는 중간 데이터입니다.

데칼 변경을 벽의 기본 재질 계산에 반영해 이후 조명에 참여하게 합니다. 화면 위에 완성된 그림을 붙이는 것과 달리, 빛을 받을 표면 특성을 수정하는 역할입니다.

투영한 변경이 기본 표면에 들어가는 과정

투영 박스와 깊이 정보는 데칼을 적용할 위치를 정하고, 데칼 재질은 바꿀 값을 정합니다. 그렇게 준비한 표면 변경을 Base Pass가 읽어 기본 표면과 결합합니다.

데칼의 투영 박스로 적용할 공간을 정합니다. 엔진은 깊이로 찾은 표면 위치가 이 범위 안에 있는지 판단하고 데칼의 텍스처 위치를 계산합니다. 데칼 재질의 변경값을 DBuffer에 준비합니다. 겹치는 데칼은 해당 경로의 합성 규칙에 따라 처리됩니다. 어떤 표면이 받는지는 수신 설정에도 달려 있습니다. 수신 재질의 Base Pass가 데칼 정보를 읽어 색·거칠기·방향 등에 반영합니다. 조명은 이렇게 바뀐 표면에 반응하므로 젖은 자국과 긁힘도 빛의 방향에 따라 다르게 보입니다.



벽에 표식을 더한 공식 전후 이미지입니다. 색뿐 아니라 거칠기와 표면 방향의 변경도 수신 재질과 조명 반응에 참여할 수 있습니다.

© Epic Games · 공식 문서 · 출처 [46]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 원저작권과 원문 출처 표시는 유지합니다

얼룩이 옆 소품에도 묻는 이유

모듈 벽의 반복을 깨고 도로 표식이나 얼룩을 추가할 때 필요합니다. 원치 않는 소품에 묻으면 텍스처색보다 투영 범위와 Receives Decals·Decal Response를 먼저 봅니다.

데칼 박스는 벽이라는 이름을 골라내는 장치가 아니라 적용할 공간입니다. 깊이가 지나치게 크면 그 안의 소품에도 같은 변경이 전달될 수 있습니다. 보이는 얼룩의 넓이와 투영 박스가 차지하는 공간은 서로 다른 조절 대상입니다.

작은 얼룩과 넓은 투영 범위의 차이

보이는 화면 면적과 중첩, 깊이 준비, 수신 재질에서 읽는 작업이 비용을 만듭니다. 이미 필요한 깊이 작업 전체를 데칼 하나의 추가 비용으로 계산하지 않습니다.

DBuffer를 이해하면 데칼을 별도 그림보다 표면의 변경으로 보게 됩니다. 원하는 외관은 재질값이 만들고, 어디에 적용될지는 투영 공간과 수신 조건이 정합니다.

참고 자료

[05] Programming with Substrate GBuffer Formats

[06] Decal Materials

05 Depth와 Early Z 가려진 표면의 계산은 언제 멈출까

벽 뒤의 소품은 화면에 보이지 않습니다. 그렇다고 그 소품과 관련된 모든 GPU 작업이 처음부터 없었던 것은 아닙니다. 올바르게 가리는 일과, 가려질 계산을 일찍 줄이는 일의 차이가 Depth와 Early Z를 이해하는 핵심입니다.

Depth Buffer는 화면 샘플의 앞뒤 관계를 판정하는 깊이 데이터를 보관합니다. Early Z는 그 깊이 판정을 재질 계산보다 일찍 적용해 가려진 후보의 계산을 생략하는 처리입니다.

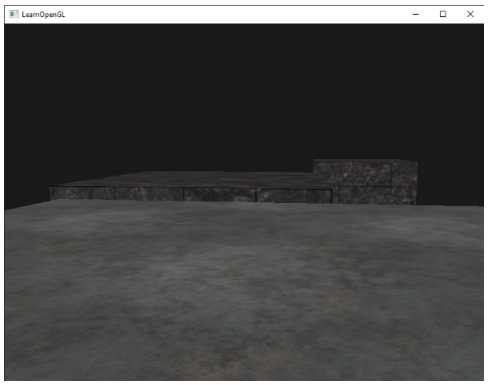
깊이 검사는 불투명 표면의 올바른 가림을 만들고, 조기 깊이 검사는 불필요한 픽셀 계산을 줄일 기회를 줍니다. Depth Prepass는 깊이를 미리 준비하는 작업으로 Early Z 자체와는 구분됩니다.

앞뒤 판정과 조기 생략은 다른 역할이다

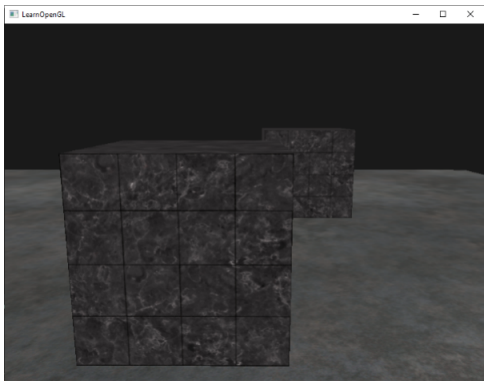
표면을 투영하면서 얻은 깊이는 이미 기록된 깊이와 비교됩니다. 그 결과에 따라 후보가 남거나 탈락하며, 가능한 조건에서는 비싼 픽셀 셰이더 계산 전에 탈락시킬 수 있습니다.

표면을 화면에 투영하면 위치별 깊이도 얻습니다. 깊이 검사로 같은 화면 위치의 앞뒤를 비교하고 설정한 규칙에 맞는 표면을 남깁니다. Prepass를 사용하는 경로에서는 먼저 깊이를 그립니다. 이후 표면 계산이 준비된 깊이를 이용할 수 있습니다. 깊이 준비 없이도 Early Z가 가능한 조건이 있으므로 두 용어를 같은 뜻으로 쓰지 않습니다. 일찍 탈락시킬 수 있는 후보는 재질 계산을 건너뛰게 할 수 있습니다. 재질이 깊이나 가시성 결정에 영향을 주는 경우에는 적용 조건이 달라집니다.

깊이 항상 통과



앞쪽 깊이로 검사



외부 깊이 검사 비교는 앞뒤 판정의 역할을 보여 줍니다. 이것만으로 Early Z의 절감 시간까지 입증되는 것은 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [47]
동일 범위의 원본을 나란히 배치 · 표제 추가 · CC BY 4.0
이용 조건

벽 하나가 줄일 수 있는 일

밀집 실내에서 벽이 뒤의 소품을 가리는 효과, 식생의 빈 영역 처리와 가림의 차이를 이해할 때 필요합니다. 최종 화면에 안 보인다는 사실만으로 모든 작업이 사라졌다고 생각하지 않습니다.

비싼 표면이 앞의 불투명 벽에 가려질 때 조기 깊이 제거가 유효하다면 그 표면의 픽셀 계산을 줄일 수 있습니다. 하지만 정점 처리와 별도 그림자 작업까지 같은 방식으로 모두 없어지는 것은 아닙니다. 화면에서 사라진 양과 절약된 전체 시간은 같지 않습니다.

깊이를 먼저 준비하는 대가

Prepass는 지오메트리를 다시 처리하는 선행 비용이 있습니다. 줄인 표면 계산이 이 비용보다 큰지 봐야 하며, 깊이 표시가 올바르다는 것과 성능 개선은 별도로 확인합니다.

깊이 버퍼는 보이는 앞면을 정하고 Early Z는 그 정보를 일찍 활용할 기회를 만듭니다. 가려짐의 정확성과 계산 절감을 나눠 이해해야 깊이 프리패스를 만능 최적화로 받아들이지 않게 됩니다.

참고 자료

[02] Render Dependency Graph

[03] Introduction to Performance Profiling and Configuration

[06] Decal Materials

[30] Visibility and Occlusion Culling

06 Overdraw 삼각형이 적은 풀이 더 느릴 수 있는 이유

풀 한 장을 사각형 두 삼각형으로 만들면 충분히 가벼워 보입니다. 하지만 작은 잎 주변에 넓은 빈 영역이 남고 수많은 카드가 겹치면 GPU가 처리할 화면의 일은 커질 수 있습니다. 폴리곤 수만으로 식생의 효율을 설명하기 어려운 이유입니다.

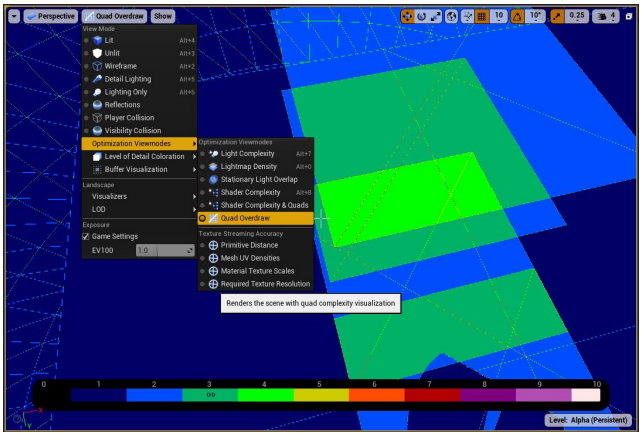
Overdraw는 화면의 같은 영역에 여러 표면 후보가 그려져 반복 처리되는 현상입니다. 앞뒤 표면의 겹침, 마스크 검사와 반투명 층 합성 등에서 생길 수 있으며 반복 처리 모두가 불필요한 것은 아닙니다.

삼각형 개수만으로 설명되지 않는 화면 단위의 작업량을 이해하는 개념입니다. 깊이로 일찍 제거할 수 없는 표면이나 여러 층이 필요한 합성에서는 중복 비용이 남기 쉽습니다.

같은 화면 영역을 반복해서 처리한다는 것

겹친 삼각형이 만드는 화면 범위에 재질과 깊이 조건이 더해지면 실제로 반복할 일이 정해집니다. 같은 겹침이라도 한 번의 계산이 무겁다면 비용은 더 커집니다.

삼각형이 화면을 덮으면 표면 후보가 생깁니다. Opaque는 불투명, Masked는 마스크로 남길 부분을 결정하고, Translucent는 뒤의 색과 섞는 표현입니다. 큰 풀 카드의 빈 부분도 어느 영역을 버릴지 마스크를 확인해야 할 수 있습니다. 잎이 작은데 카드가 크면 잎 바깥의 넓은 영역까지 후보를 만듭니다. 여러 카드나 투명 층이 겹치면 비슷한 화면 위치를 반복 처리합니다. 불투명 깊이 제거, 마스크와 반투명 조건에 따라 실제 실행과 생략의 정도가 달라집니다.



외부 Quad Overdraw 진단 화면은 작은 화면 묶음의 처리 비효율을 드러냅니다. 진단색은 GPU 실행 시간 자체가 아닙니다.

Android Developers · Google · 출처 [48]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · CC BY 4.0
이용 조건

잎이 없는 공간도 공짜는 아니다

풀·나뭇잎·연기·머리카락에서 메시의 빈 공간과 겹침을 줄일 때 필요합니다. 윤곽을 다듬어 삼각형을 늘려도 화면 낭비가 줄어 전체가 빨라지는 경우를 설명합니다.

사각형 카드의 대부분이 비어 있다면 그 영역에서도 마스크를 확인하는 일이 필요할 수 있습니다. 잎의 윤곽을 따라 메시를 다듬은 쪽은 삼각형이 늘지만 불필요한 후보 영역은 줄어듭니다. 어느 쪽이 유리한지는 화면 크기와 실제 병목에 달려 있습니다.

진단색이 곧 GPU 시간은 아니다

겹치는 면적·층 수·재질 계산이 함께 비용을 만듭니다. Shader Complexity와 Quad Overdraw는 원인 후보를 보는 진단이며 실제 실행 시간과 같은 단위가 아닙니다.

식생의 효율은 적은 삼각형만으로 완성되지 않습니다. 화면에서 차지하는 범위와 겹친 층, 빈 공간까지 함께 보아야 형상과 표면 계산 사이의 균형을 잡을 수 있습니다.

참고 자료

[02] Render Dependency Graph

[07] Viewport Modes

[19] Nanite Virtualized Geometry

07 PBR 조명이 바뀌어도 재료답게 보이는 표면

한 라이팅에서는 그럴듯했던 금속이 다른 장면에서 플라스틱처럼 보이기도 합니다. 이런 차이를 무조건 텍스처색의 문제로 돌리면 수정은 끝나지 않습니다. PBR은 재료에 속한 특성과 장면의 빛을 분리해서 다루는 기준을 제공합니다.

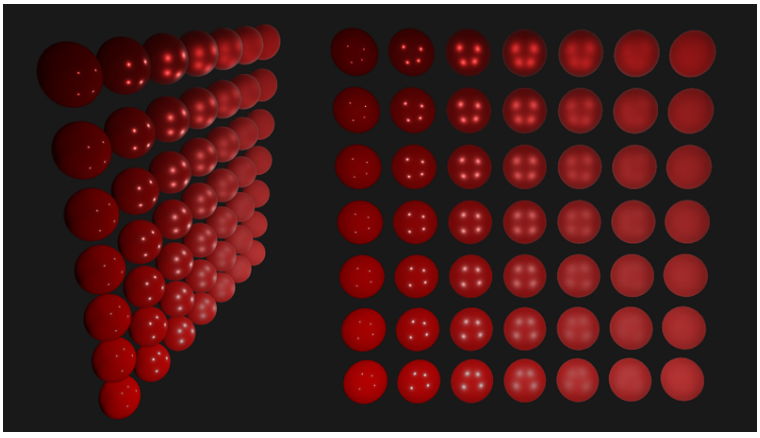
PBR은 Physically Based Rendering으로, 물질과 빛의 상호작용을 물리적으로 그럴듯하게 근사하는 렌더링 접근입니다. 아트 제작에서는 재료의 특성을 입력하고 장면의 빛에 일관되게 반응하도록 만드는 기준이 됩니다.

재질의 고유 특성과 특정 장면의 조명을 분리합니다. 한 환경에서만 그럴듯한 색을 그리는 대신, 다른 조명에서도 금속·도장·돌의 구분이 유지되도록 합니다.

재질 입력이 빛에 대한 반응이 되는 과정

기본 색과 금속성, 거칠기, 표면 방향은 재료가 빛에 반응할 조건을 제공합니다. 셰이딩 모델은 이 조건을 장면의 빛과 결합해 카메라 방향으로 나오는 빛을 계산합니다.

재료를 금속·비금속과 표면 상태로 구분합니다. 일반 비금속의 Base Color는 주로 확산 반사색에, 금속의 Base Color는 금속 반사색에 관여합니다. Roughness로 반사가 퍼지는 정도를, 법선으로 미세 표면 방향을 정합니다. Specular 등 입력의 역할은 사용하는 모델에 맞게 적용하며, 밝기 조정용 만능 값으로 쓰지 않습니다. 렌더러가 재료 반응에 광원·환경과 그림자를 결합합니다. 그래서 재료 입력은 같아도 장면의 최종 외관은 달라질 수 있습니다. 조명을 바꿔 재료 구분이 유지되는지 확인합니다.



외부 재질 구 배열은 금속성과 거칠기에 따른 빛 반응의 차이를 보여 줍니다. 배치와 조명 조건도 외관에 관여합니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [40]
원본 전체와 비율 유지 · CC BY 4.0
이용 조건

페인트와 드러난 철은 다른 재료다

도장 아래 드러난 철, 녹과 오염을 재료별로 나눌 때 필요합니다. Base Color에 특정 방향의 하이라이트를 그려 넣으면 광원이 움직여도 남으므로 물질 흔적과 조명 흔적을 구분합니다.

도장면과 녹은 비금속이고 벗겨진 철은 금속입니다. 단순히 밝고 어두운 무늬로 구분하면 조명 변화에서 재료의 차이가 약해질 수 있습니다. 기본 색에 특정 광원의 하이라이트까지 그려 넣은 경우에는 라이트가 이동해도 그 흔적만 표면에 남습니다.

물리 기반이라는 이름이 비용을 정하지는 않는다

PBR이라는 이름이 고정 성능 비용을 정하지는 않습니다. 사용하는 텍스처, 혼합 구조, 셰이딩 모델과 화면 점유율이 실제 계산·메모리 부담을 만듭니다.

PBR은 현실 사진을 그대로 붙이는 요령보다 재료의 고유 반응을 분리해 작성하는 방식에 가깝습니다. 재질이 어떤 빛을 받아도 같은 색으로 보여야 하는 것은 아니지만, 재료의 성격은 이어져야 합니다.

참고 자료

[08] Physically Based Materials

[09] Using Fresnel

08 BRDF 하이라이트의 모양은 어디에서 오는가

넓은 하이라이트를 보고 재질이 거칠다고 판단하기 쉽습니다. 그러나 넓은 광원을 비춘 매끈한 표면에서도 비슷한 인상이 나올 수 있습니다. BRDF는 빛의 방향과 표면의 성질이 반사의 모양으로 이어지는 관계를 설명합니다.

BRDF는 들어오는 빛의 방향에 대해 특정 방향으로 얼마나 반사되는지를 나타내는 함수입니다. 표면에서 바로 일어나는 반사를 설명하며, 표면 내부를 통과하는 모든 빛 전달을 뜻하지는 않습니다.

재질 속성의 숫자가 실제 반사 분포로 바뀌는 연결입니다. 거칠기·표면 방향과 빛의 방향이 하이라이트의 폭과 세기를 바꾸는 이유를 설명합니다.

들어온 빛이 어느 방향으로 나가는가

빛과 관찰 방향, 법선과 재료 특성이 BRDF의 반응을 정합니다. 렌더러는 방향별 반사 반응에 들어오는 빛의 양을 결합해 표면에서 나오는 빛을 구합니다.

빛이 표면에 어떤 방향으로 도달하는지와 카메라가 어느 방향에서 보는지 정합니다. 같은 재료도 이 두 방향의 관계가 바뀌면 반사가 달라집니다. 미세면 모델은 작은 표면 방향들의 분포로 거친 표면을 근사합니다. 거칠기가 높으면 반사 방향이 넓게 퍼지고, 표면 방향은 반사가 나타나는 위치를 바꿉니다. 여러 방향에서 들어오는 광원의 기여를 모아 관찰 방향의 색을 얻습니다. 넓은 광원도 넓은 하이라이트를 만들 수 있으므로 재질과 광원의 영향을 구분합니다.



외부 미세면 개념도에서 거친 표면은 다양한 방향으로 빛을 분산시킵니다. 실제 하이라이트에는 광원의 방향 범위도 반영됩니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [41]
원본 전체와 비율 유지 · CC BY 4.0
이용 조건

거칠기와 광원 크기가 비슷하게 보일 때

젖은 돌·거친 콘크리트·도장 금속의 반사를 조절할 때 필요합니다. 같은 넓이의 하이라이트도 거칠기와 광원 크기라는 다른 원인으로 생길 수 있습니다.

거칠기는 재료의 반사 분포를 바꾸고, 광원의 크기는 들어오는 빛의 방향 범위를 바꿉니다. 둘 다 하이라이트를 넓힐 수 있지만 같은 조절은 아닙니다. 재료감을 조명 크기로만 맞추면 그 재질을 다른 환경으로 옮겼을 때 의도한 반응이 유지되지 않을 수 있습니다.

반사 모델과 실제 계산 비용

사용하는 모델의 평가, 광원과 화면 범위가 비용에 관여합니다. 거칠기 숫자 하나를 바꾸는 것과 복잡한 층·텍스처 계산을 추가하는 일은 구분합니다.

하이라이트는 재질값 하나의 그림이 아니라 표면과 빛이 만난 결과입니다. BRDF를 이해하는 목적은 수식을 외우기보다 어떤 외관을 재질에서 만들고 어떤 외관을 광원에서 만들지 구분하는 데 있습니다.

참고 자료

[08] Physically Based Materials

[10] Overview of Substrate Materials

[31] Real Shading in Unreal Engine 4 · Brian Karis

09 Fresnel 물과 유리의 반사가 각도에 따라 달라지는 이유

물웅덩이를 위에서 내려다볼 때와 낮은 시선으로 볼 때 반사의 비중은 다릅니다. 같은 표면인데도 보는 방향에 따라 인상이 달라지는 데에는 Fresnel이라는 경계면의 빛 반응이 관여합니다.

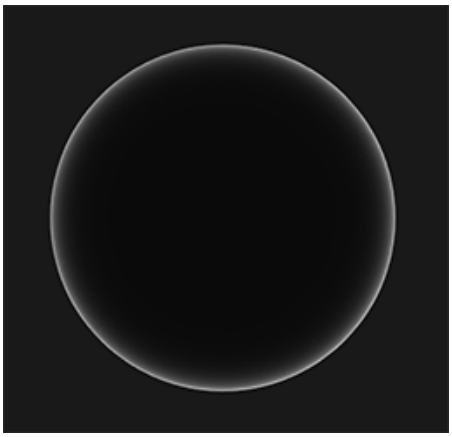
Fresnel은 두 물질의 경계에서 반사되는 빛의 비율이 각도와 물질 특성에 따라 달라지는 현상입니다. 일반 비금속 경계에서는 스치는 각도에서 반사가 강해지는 반응이 대표적입니다.

BRDF와 투과 모델 안에서 반사와 투과의 비중을 정하는 데 쓰입니다. 물·유리·코팅이 보는 각도에 따라 다르게 보이는 물리적 근거입니다.

경계에서 반사되는 빛의 비율

물질 경계의 특성과 입사각이 반사율을 바꿉니다. 이 반사율은 광원과 환경, 거칠기와 함께 작용하므로 각도 반응 하나만으로 화면의 밝기를 모두 설명할 수는 없습니다.

정면과 비스듬한 입사의 반사율을 구분합니다. F0는 정면 입사에서의 반사율로, 물질 경계의 특성을 표현하는 기준 중 하나입니다. 각도 변화에 따른 반사율을 계산해 표면 반응에 반영합니다. 거친 표면에서는 미세면 방향들이 섞이므로 단순한 화면 가장자리 선 하나로 나타나는 현상은 아닙니다. 렌더러가 반사할 환경과 빛을 결합합니다. 한편 UE의 Fresnel 노드는 시선과 표면 방향으로 마스크를 만들 수 있어, 물리 반사 외의 림 효과에도 사용됩니다.



외부 Fresnel 마스크는 각도 의존성을 이해하는 데 쓰는 표현입니다. 발광에 연결한 마스크와 물질의 실제 반사율은 구분됩니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [41]
원본 전체와 비율 유지 · CC BY 4.0
이용 조건

물리 반사와 가장자리 발광을 구분하기

물과 유리의 반사를 자연스럽게 조정하거나, 별도의 가장자리 발광을 설계할 때 필요합니다. 기본 PBR 반사에 포함된 각도 반응과 추가한 아트 효과를 구분해야 중복 보정을 피할 수 있습니다.

PBR 반사에는 이미 각도에 따른 반응이 포함됩니다. 여기에 Fresnel 노드를 발광으로 더하면 반사율을 조절하는 것과 다른 연출이 됩니다. 주변 환경이 어두워도 가장자리가 밝아질 수 있기 때문에, 두 효과는 비슷해 보여도 빛을 만드는 원인이 다릅니다.

유리 전체의 비용을 마스크 하나로 설명할 수는 없다

기본 재질의 각도 반응과 추가 텍스처·발광·반투명 기능의 비용은 다릅니다. Fresnel 노드 하나의 외관을 전체 유리 렌더링 비용과 동일시하지 않습니다.

Fresnel을 이해하면 각도에 따라 달라지는 자연스러운 반사와 의도적으로 추가한 림 효과를 구분할 수 있습니다. 물과 유리의 재료감은 가장자리에 밝은 선을 넣는 것보다 더 넓은 관계 속에 있습니다.

참고 자료

[08] Physically Based Materials

[09] Using Fresnel

[10] Overview of Substrate Materials

[31] Real Shading in Unreal Engine 4 · Brian Karis

10 셰이더와 머티리얼 그래프는 어떻게 화면 계산이 될까

머티리얼 에디터에서 선을 연결하는 일은 결국 GPU가 실행할 계산을 작성하는 일입니다. 하지만 그래프 전체를 픽셀 셰이더 하나라고 생각하면 풀의 움직임과 표면색, 화면 효과가 어디에서 달라지는지 놓치기 쉽습니다.

셰이더는 GPU에서 실행하는 계산 프로그램입니다. 머티리얼은 표면이 어떻게 보이고 빛에 반응할지 작성하는 엔진의 제작 체계이며, 머티리얼 그래프는 셰이더 계산으로 변환되는 입력 중 하나입니다.

지오메트리·텍스처·재질 속성을 실제 화면 계산으로 연결합니다. 정점의 위치를 바꾸는 일, 표면 값을 구하는 일과 화면 이미지를 처리하는 일은 서로 다른 단계의 프로그램으로 실행될 수 있습니다.

작성한 그래프에서 실행 프로그램까지

프로그램이 받는 데이터에는 정점 속성과 텍스처, 파라미터와 장면 정보가 있습니다. 어떤 단계의 셰이더인지에 따라 변환된 위치나 표면 특성, 색 또는 다른 계산 데이터를 만듭니다.

그래프를 작성하면 엔진이 사용 경로와 설정에 맞는 코드를 구성하고 컴파일합니다. 컴파일은 실행할 프로그램으로 바꾸는 준비 과정입니다. 설정 조합에 따라 서로 다른 셰이더 순열이 필요할 수 있습니다. 정점 셰이더는 위치와 속성 등을 처리합니다. WPO는 기존 정점 위치에 이동량을 더하는 머티리얼 입력입니다. 같은 함수라도 정점 배치가 다르면 표현할 수 있는 굽힘이 달라집니다. 픽셀 셰이더는 표면 후보에서 텍스처와 재질 계산을 수행합니다. 컴퓨트 셰이더는 그림이나 버퍼를 일반적인 병렬 계산으로 처리하는 데 쓰입니다. 그래프의 노드 전체가 픽셀 셰이더 하나와 항상 일대일로 대응하지는 않습니다.



회색은 원래 위치 · 청록은 같은 높이별 이동 함수를 적용한 위치

같은 높이별 변위 함수라도 정점 배치가 다르면 표현할 수 있는 굽힘이 달라집니다. 정점 계산의 한 적용을 보여 주는 직접 제작 도식입니다.

풀의 굽힘과 표면의 무늬는 다른 계산이다

식생 변형은 정점, 표면 혼합은 주로 재질 평가, 후처리는 화면 계산을 바꿉니다. 어떤 입력이 어느 일을 늘리는지 알아야 텍스처 해상도로 형상 문제를 고치려는 혼동을 피할 수 있습니다.

WPO는 기존 정점의 위치를 움직입니다. 긴 카드에 중간 정점이 없다면 텍스처를 아무리 정교하게 만들어도 중간이 휘는 실루엣을 얻기 어렵습니다. 반대로 표면의 미세한 무늬는 정점을 늘리는 것만으로 생기지 않습니다. 데이터의 세밀함도 필요한 계산에 맞게 배분해야 합니다.

실행 비용과 준비 시간은 따로 움직인다

정점·픽셀 수, 읽는 데이터와 실행 연산이 GPU 비용을 만듭니다. 컴파일 대기와 쿠킹 준비 시간은 실행 비용과 별개의 지표입니다. 노드 수만으로 실제 실행 시간을 단정하지 않습니다.

셰이더는 GPU의 프로그램이고 머티리얼은 그중 표면 반응을 작성하는 체계입니다. 이 관계가 보이면 노드 개수보다 어떤 단계의 어떤 데이터와 작업을 늘리는지로 그래프를 읽을 수 있습니다.

참고 자료

[02] Render Dependency Graph

[11] Material Editor UI

[12] Material Inputs

[62] Graphics Programming Overview · Epic

11 Substrate 재료를 섞는 것과 층을 쌓는 것의 차이

거친 페인트 위에 매끈한 코팅이 올라간 표면은 거칠기 두 값을 평균한 표면과 다릅니다. 빛 일부는 코팅에서 반사되고 나머지는 아래 재료까지 내려갔다 돌아옵니다. Substrate는 이런 재료의 구성과 관계를 표현하는 머티리얼 체계입니다.

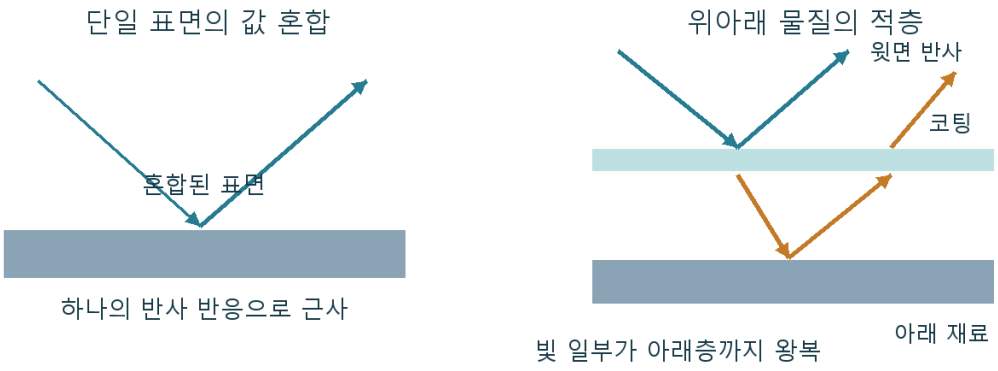
Substrate는 UE에서 물질의 빛 반응을 모듈식으로 작성하고 혼합·적층하는 머티리얼 프레임워크입니다. 하나의 코팅 효과 이름이 아니라 여러 물질 표현을 구성하는 체계입니다.

고정된 모델의 제한된 입력만 고르는 대신 물질 단위와 관계를 연결해 복합 표면을 표현합니다. 도장·코팅·물기·산란 같은 재료의 구성을 설명할 수 있게 합니다.

물질 단위와 관계로 표면을 구성한다

Slab 같은 물질 단위의 특성과 연산자의 연결이 표면 구성을 만듭니다. 렌더러는 이를 처리 가능한 표현으로 준비한 뒤 조명에서 빛 반응을 평가합니다.

Slab은 반사·투과·산란 같은 상호작용을 표현하는 물질 단위입니다. BSDF는 경계에서 빛이 어느 방향으로 반사·투과되는지 나타내는 분포 함수의 범주입니다. 혼합은 재료의 비중을 결합하고, 수직 적층은 빛이 위아래 물질을 거치는 관계를 표현합니다. 아래층의 반응은 윗층을 통과하는 빛의 영향을 받으므로 단순한 색·거칠기 평균과 다를 수 있습니다. 구성한 표현은 플랫폼과 예산에 맞는 단순화·저장·평가를 거칩니다. 작성한 층의 의도와 실제 실시간 결과를 여러 조명과 각도에서 확인합니다.



직접 제작한 빛 경로 개념도 · 실제 추적·단순화 구현을 그대로 묘사한 그림은 아닙니다.

단일 표면의 값 혼합과 위아래 물질을 통과하는 빛의 관계를 나눈 직접 제작 도식입니다. 실제 런타임 추적·단순화를 그대로 재현한 그림은 아닙니다.

아래 페인트와 위 코팅이 함께 보여야 할 때

광택 코팅 아래 거친 페인트, 젖은 돌, 재료 경계처럼 서로 다른 반응을 함께 만들 때 필요합니다. 차이가 읽히지 않는 원거리 표면까지 복잡한 구성을 유지할 필요는 없습니다.

윗층의 선명한 반사와 아래층의 넓은 반응이 동시에 읽히는 표면이라면 하나의 거칠기 평균으로 두 인상을 맞추기 어렵습니다. 반면 멀리 보이는 소품에서는 그 차이가 거의 사라질 수 있습니다. 복잡한 물질 구성이 필요한지는 화면에서 읽히는 반응에 달려 있습니다.

작성한 층과 실시간 평가의 관계

물질의 복잡도·저장량·평가 경로와 단순화가 비용에 관여합니다. Slab 개수만으로 고정된 바이트나 시간을 약속하지 않고 대상 환경의 실제 결과를 확인합니다.

Substrate의 의미는 무조건 층을 더 쌓을 수 있다는 데 있지 않습니다. 원하는 재료 반응을 어떤 단위와 관계로 구성할지 명확하게 만들고, 필요한 표현에 맞춰 복잡도를 배분하는 데 있습니다.

참고 자료

[05] Programming with Substrate GBuffer Formats

[06] Decal Materials

[10] Overview of Substrate Materials

12 Translucency 유리가 불투명 벽보다 까다로운 이유

불투명 벽 앞에서는 뒤의 색을 잊어도 되지만 유리 앞에서는 그럴 수 없습니다. 유리 자체의 반사와 뒤의 장면, 여러 겹의 표면을 함께 다뤄야 최종 색이 나옵니다. 반투명 표현의 어려움은 이 합성 관계에서 시작합니다.

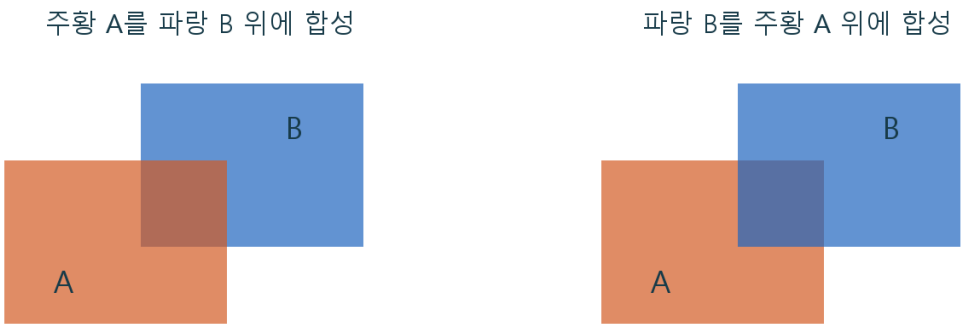
Translucency는 뒤쪽 장면의 기여를 남기며 표면의 빛 반응과 배경을 결합하는 반투명 표현입니다. 알파 합성, 조명과 굴절이 함께 사용될 수 있지만 각각의 역할은 다릅니다.

앞면 하나만으로 색이 정해지는 불투명 표면과 달리 여러 표면의 색을 합쳐 화면을 구성합니다. 유리·물·연기가 배경과 어울리도록 만드는 렌더링 영역입니다.

앞 표면 하나로 끝나지 않는 색

반투명 표면의 색과 불투명도, 조명 반응은 뒤쪽 장면과 결합됩니다. 합성한 색은 다시 앞의 다른 층에 쓰일 수 있고, 굴절이 있으면 배경을 읽는 위치나 추적 경로도 달라집니다.

반투명 표면의 재질과 조명을 계산합니다. 값이 작아진 Base Color와 낮은 알파는 같은 조절이 아니며 어떤 합성·셰이딩 모드를 쓰는지 먼저 정합니다. 기본 알파 합성에서는 앞 색과 뒤 색의 기여를 섞습니다. 순서를 바꾸면 결과가 달라질 수 있으므로 깊이 검사만으로 끝내지 않고 표면의 정렬과 합성 순서가 필요합니다. 굴절이 있다면 배경을 다른 위치에서 읽거나 지원 경로로 추적합니다. 합성 정렬의 오류와 화면 밖 배경을 얻지 못하는 굴절 오류는 다른 원인입니다.



직접 계산한 알파 합성 개념도 · 같은 두 색과 알파 0.7에서 합성 순서만 변경

기본 알파 연산으로 두 색과 알파를 유지하고 순서만 바꾼 비교입니다. 겹친 색이 달라지는 것은 합성 순서가 결과에 참여하기 때문입니다.

겹침 오류와 굴절 오류는 다르다

겹친 유리창·교차하는 판·연기에서 정렬과 배경 누락을 구분할 때 필요합니다. 외관이 이상하다고 재질색부터 바꾸기보다 표면 구조와 합성 경로를 봅니다.

기본 알파 합성은 두 색의 순서를 바꾸면 겹친 결과도 달라질 수 있습니다. 서로 교차하는 판은 물체 단위의 정렬만으로 모든 픽셀의 앞뒤를 맞추기 어렵습니다. 여기에 굴절까지 더하면 배경의 어느 위치를 읽는지라는 문제가 따로 생깁니다.

겹친 층만큼 남는 화면 작업

넓은 화면 면적·여러 층·조명·굴절 계산이 비용을 만듭니다. 뒤쪽 층의 색도 필요하므로 불투명 앞면처럼 모두 생략하기 어렵습니다.

반투명은 색을 옅게 만드는 설정 하나가 아니라 뒤의 장면을 포함해 화면을 구성하는 방식입니다. 합성 순서와 굴절을 나눠 이해하면 재질색으로 해결하기 어려운 오류가 보이기 시작합니다.

참고 자료

[13] Lit Translucency
[14] Using Refraction

13 SSS 왁스와 돌의 내부에서 일어나는 빛의 변화

왁스의 얇은 가장자리는 역광에서 부드럽게 빛나지만 두꺼운 몸통은 다른 밀도로 보입니다. 표면의 색만으로 설명하기 어려운 차이입니다. 빛이 재료 안을 거쳐 나오는 과정인 SSS가 이 외관에 관여합니다.

SSS는 Subsurface Scattering으로, 빛이 표면 안으로 들어가 내부에서 흩어지고 흡수된 뒤 다시 나오는 현상입니다. 표면에서 즉시 반사되는 빛과 구분되는 전달입니다.

피부·왁스·대리석 등에서 부드러운 음영과 내부 빛 전달에 의한 색을 표현합니다. 표면 반사만으로는 만들기 어려운 재료의 밀도감을 담당합니다.

표면에서 끝나지 않는 빛의 경로

산란과 흡수 특성에 두께·스케일과 조명이 더해지면 내부 전달의 조건이 정해집니다. 실시간 모델은 그 조건을 근사해 표면에서 나오는 빛의 색과 분포에 반영합니다.

빛 일부가 표면 안으로 들어갑니다. 물질은 색에 따라 빛을 다르게 흡수하고, 내부 산란으로 진행 방향을 바꿉니다. 지나온 거리와 재료 특성에 따라 나오는 빛이 달라집니다. 그래서 같은 재료라도 얇은 가장자리와 두꺼운 몸통의 역광 반응이 다를 수 있습니다. 실시간 모델은 이 전달을 여러 방식으로 근사합니다. 모든 광선을 물질 내부에서 실제 추적하는 것은 아니며, 필요한 입력과 근사 조건은 선택 모델에 달려 있습니다.



같은 물질이어도 빛이 지나온 거리와 산란에 따라 나오는 빛이 달라집니다.
직접 제작 현상 개념도 · 특정 UE 산란 모델이 이 경로를 그대로 추적한다는 뜻은 아닙니다.

같은 물질에서도 내부 경로가 달라질 수 있음을 보여 주는 개념도입니다. 특정 UE 산란 모델의 실제 광선 추적을 재현하지는 않습니다.

알파를 낮추는 것과 내부 산란은 다르다

왁스의 얇은 부분, 피부의 부드러움과 일부 돌의 투광감을 만들 때 필요합니다. 알파를 내려 배경을 비추는 처리와 내부 빛 전달을 혼동하지 않습니다.

왁스의 역광을 만들기 위해 불투명도만 낮추면 배경이 그대로 비치면서 유리 같은 인상이 생길 수 있습니다. 내부 산란은 뒤의 사진을 섞는 대신 재료 안의 빛 전달을 표현합니다. 같은 색이라도 두께와 흡수, 산란 거리가 달라지면 밀도감이 달라지는 이유입니다.

모델과 스케일이 외관과 비용을 바꾼다

선택한 산란 모델, 화면 범위와 추가 계산이 비용에 관여합니다. 재료의 스케일과 두께를 틀리게 두고 산란색만 보정하면 다른 조명에서 의도가 무너질 수 있습니다.

SSS는 재료를 흐릿하게 만드는 효과보다 빛이 내부에서 이동하는 현상에 대한 근사입니다. 표면 반사와 내부 전달을 구분해야 피부·왁스·돌의 서로 다른 밀도를 설계할 수 있습니다.

참고 자료

[10] Overview of Substrate Materials

[15] Subsurface Profile Shading Model

14 Draw Call 소품 하나가 여러 그리기 작업이 되는 이유

레벨의 Actor 수가 적다고 그리기 명령도 적은 것은 아닙니다. 재질 슬롯이 많은 소품 하나는 여러 작업으로 나뉠 수 있고, 같은 형상도 그림자와 표면을 위해 다시 처리될 수 있습니다. Draw Call은 그 연결을 이해하는 단위입니다.

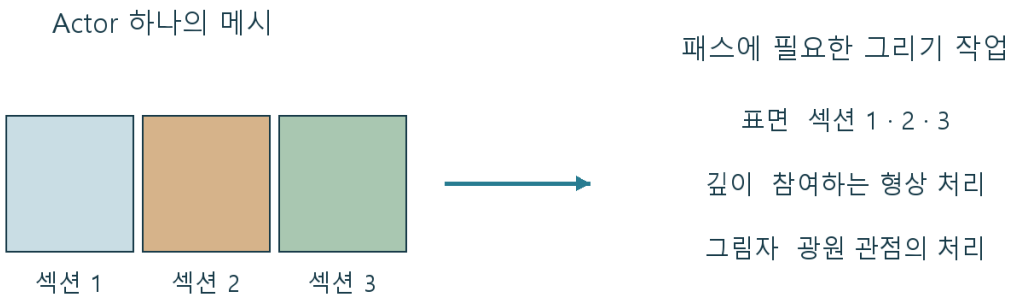
Draw Call은 GPU에 정해진 상태와 지오메트리로 그리기 작업을 실행하도록 제출하는 명령입니다. Actor는 레벨의 오브젝트 단위이고 Draw Call은 실행 작업의 단위이므로 개수가 같지 않습니다.

CPU가 준비한 장면을 GPU의 그리기 작업으로 연결합니다. 표면을 어떻게 나누고 어떤 재질과 패스에 참여시키는지가 제출 부담을 만드는 연결 지점입니다.

물체 수에서 명령 수로 넘어가는 사이

그릴 메시 구역과 재질, 패스의 상태가 실행 작업으로 준비됩니다. 렌더러는 호환되는 일을 묶어 제출할 수 있고, GPU는 전달된 조건으로 정점과 화면 처리를 수행합니다.

렌더러가 보이는 메시 섹션을 확인합니다. 섹션은 서로 다른 재질 등을 적용할 수 있는 메시 구역이며, 구역별 상태가 다르면 별도 작업이 필요할 수 있습니다. 깊이·기본 표면·그림자처럼 필요한 패스마다 참여 작업을 만듭니다. 같은 메시가 여러 패스에 쓰이므로 물체 하나가 한 번의 명령으로 끝나지 않을 수 있습니다. 호환되는 작업은 배칭이나 인스턴싱으로 묶을 수 있습니다. 실제 제출 수는 자동 묶음과 렌더 경로의 영향을 받으므로 Actor·섹션·실행 명령을 구분해 셉니다.



섹션과 패스가 제출 작업을 나눕니다. 자동 묶음과 경로에 따라 실제 명령 수는 달라집니다.

하나의 Actor와 여러 메시 섹션, 패스별 작업의 관계를 보여 주는 도식입니다. 자동 묶음과 경로에 따라 실제 명령 수는 달라집니다.

메시를 합쳐도 제출이 기대만큼 줄지 않을 때

모듈 건물의 슬롯 수·아틀라스·공유 재질·메시 합체를 결정할 때 필요합니다. 합체로 오브젝트를 줄여도 섹션이 남거나 가시성이 나빠지면 기대한 이득이 없을 수 있습니다.

나무·금속·흙을 여러 섹션으로 나눈 소품은 하나의 Actor 안에서도 서로 다른 상태의 그리기 작업을 만들 수 있습니다. 오브젝트를 합쳐도 그 구역이 남으면 제출이 그대로 남을 수 있고, 너무 크게 합친 형상은 가려진 부분을 따로 빼기 어려워집니다.

호출 수와 프레임 시간은 같은 지표가 아니다

명령과 상태를 준비하는 CPU 비용, GPU의 실제 표면 비용은 다릅니다. 호출 수가 줄어도 픽셀 계산이 지배적이면 총 시간이 거의 그대로일 수 있습니다.

Draw Call은 물체의 개수보다 GPU에 전달할 일의 구성을 보여 줍니다. 슬롯과 합체를 결정할 때 제출 부담과 공간별 가시성을 함께 고려해야 숫자 하나만 줄이는 최적화를 피할 수 있습니다.

참고 자료

[03] Introduction to Performance Profiling and Configuration

[16] Mesh Drawing Pipeline

15 Instancing 반복되는 배치에서 무엇을 공유할까

울타리 기둥 수백 개는 형상보다 위치가 다릅니다. 그 공통점과 차이를 데이터 구조에 반영하는 방식이 인스턴싱입니다. 반복되는 물체를 각각 완전히 독립된 대상으로 다루지 않아도 되는 이유가 여기에 있습니다.

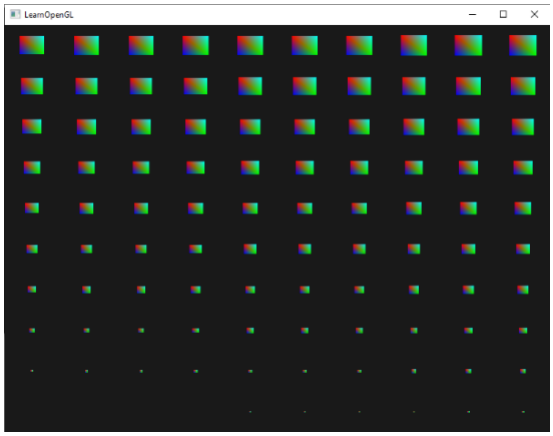
Instancing은 공통 메시 같은 데이터를 공유하면서 위치·회전·크기 등의 배치별 정보를 따로 두어 여러 복사본을 처리하는 방식입니다. 같은 형상을 반복하는 데 적합한 데이터·실행 구조입니다.

반복 오브젝트를 개별적으로 관리하고 제출하는 부담을 줄일 수 있습니다. 여러 복사본이 보이는 사실 자체를 없애는 것이 아니라 중복 데이터와 준비를 공유하는 역할입니다.

공통 형상과 배치별 정보를 나누는 구조

공통 메시와 재질에 각 배치의 Transform이 더해져 여러 위치의 복사본이 됩니다. 형상 데이터는 공유해도 각 위치에서 보이는 삼각형과 픽셀은 여전히 처리해야 합니다.

공통 형상과 재질을 준비합니다. Transform은 위치·회전·크기의 변환 정보이며 같은 형상을 장면의 다른 위치에 배치하게 합니다. ISM은 반복 배치를 하나의 Instanced Static Mesh Component에서 관리합니다. 인스턴스별 색 같은 변주는 공유 재질에서 별도의 배치별 데이터로 읽을 수 있습니다. 렌더러가 인스턴스의 가시성과 실행 조건을 고려해 처리합니다. HISM은 공간 계층을 사용하는 확장이며 갱신·LOD·컬링 조건에 따라 적합성이 달라집니다.



외부 인스턴스 배열은 공통 형상에 배치별 정보를 더하는 한 사례입니다. 화면의 복사본을 모두 한 번만 셰이딩한다는 뜻은 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [52]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · CC BY 4.0
이용 조건

같은 기둥에 다른 색을 주는 방법

울타리 기둥·식생·가로등·군집 소품처럼 같은 형상이 반복될 때 필요합니다. 배치마다 독립 재질을 만드는 대신 공통 데이터와 개별 변주를 나눠 설계합니다.

인스턴스별 색 변화는 메시와 재질을 매번 새로 만들지 않고 배치별 데이터로 전달할 수 있습니다. 공통 재질이 그 값을 읽으면 형상과 계산 구조를 공유하면서도 변주가 생깁니다. 반복 배치의 효율과 화면의 다양성이 반드시 반대인 것은 아닙니다.

공유해도 복사본의 픽셀은 남는다

관리·제출 부담을 줄일 수 있지만 정점·픽셀·그림자와 위치 갱신은 남습니다. 자동 묶음과 Nanite 조건에 따라 제출 이득은 달라질 수 있습니다.

인스턴싱의 중심은 복사본을 없애는 것이 아니라 공유할 부분을 정확하게 나누는 데 있습니다. 공통 데이터와 개별 상태를 분리하면 배경의 반복을 관리하는 방식도 달라집니다.

참고 자료

[16] Mesh Drawing Pipeline

[17] Instanced Static Mesh Component

16 LOD 멀리서도 남겨야 하는 디테일은 무엇일까

먼 난간의 나사는 보이지 않아도 기둥의 간격과 윗선은 남습니다. 에셋의 모든 면을 같은 비율로 줄이면 이 차이를 놓치기 쉽습니다. LOD는 단순한 폴리곤 감소보다 화면에서 필요한 정보를 고르는 문제입니다.

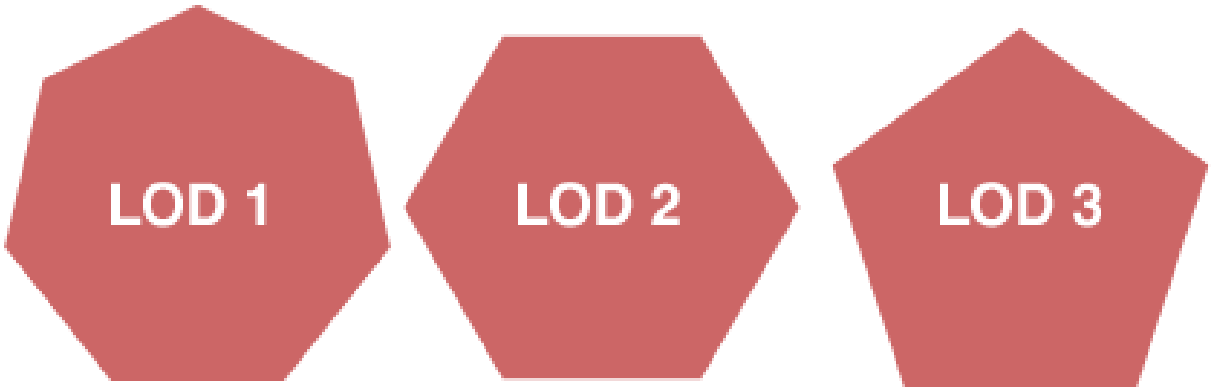
LOD는 Level of Detail로, 화면에서 필요한 정도에 맞춰 표현의 세부 수준을 바꾸는 방식입니다. 전통적인 Static Mesh LOD는 미리 만든 대체 메시를 선택해 사용합니다.

화면에 거의 기여하지 않는 디테일에 쓰는 작업을 줄이면서 필요한 외관을 유지합니다. 단순히 먼 물체를 대충 만드는 것이 아니라 화면 정보에 자원을 배분하는 원리입니다.

거리가 아니라 화면에 남는 크기를 기준으로

단계별 메시와 화면 크기 조건에 따라 사용할 표현이 선택됩니다. 물체의 실제 크기와 카메라 시야각도 화면 점유율을 바꾸므로 거리 하나만으로 세부를 정하지 않습니다.

원본을 기준으로 작은 부품과 면을 줄인 대체 표현을 준비합니다. 실루엣·큰 간격·그림자에 중요한 형태를 먼저 구분합니다. 렌더러가 화면 점유율 등의 기준으로 표현 단계를 선택합니다. 같은 거리라도 큰 물체와 작은 물체 또는 다른 FOV에서는 필요한 디테일이 다릅니다. 카메라 이동으로 단계가 바뀝니다. 두 표현의 윤곽·노멀·재질 차이가 크면 전환 순간 튀므로 정지 외관뿐 아니라 이동과 그림자도 확인합니다.



외부 LOD 그림에서는 단순화에 따라 윤곽이 각지게 바뀝니다. 원본의 LOD 1·2·3 표기는 UE에서 흔히 시작하는 LOD0 번호와 다릅니다.

Sirotk · Wikimedia Commons · 출처 [42]

원본 전체와 비율 유지 · CC BY-SA 3.0

이용 조건

나사를 없애도 윤곽은 남겨야 한다

난간의 나사와 베벨, 먼 건물의 장식처럼 플레이 거리에서 읽히지 않는 부분을 줄일 때 필요합니다. 비율로 일괄 감소하기보다 남길 시각 정보를 먼저 정합니다.

카메라가 멀어지면 작은 베벨과 나사는 몇 픽셀 안으로 사라집니다. 반면 상단 실루엣을 크게 바꾸면 전환 순간 건물의 형태가 튈 수 있습니다. 단순화의 우선순위는 부품의 실제 크기보다 플레이 화면에서 맡는 역할에 달려 있습니다.

줄인 삼각형이 전체 시간을 줄이는 조건

정점·삼각형·섹션과 재질을 줄일 수 있지만 픽셀 비용이 같으면 총 이득은 작을 수 있습니다. 전환 중 두 표현이 처리되는 조건도 별도로 확인합니다.

좋은 LOD는 원본을 균등하게 덜어낸 형상이 아니라 멀리서 읽힐 정보를 남긴 표현입니다. 화면 크기와 이동 중 전환을 기준으로 보면 무엇을 생략할지 더 분명해집니다.

참고 자료

[17] Instanced Static Mesh Component

[18] Optimizing LOD Screen Size Per Platform

17 Nanite 원본 디테일과 화면의 삼각형 수가 다른 이유

높은 디테일의 절벽을 가져왔다고 그 원본의 모든 삼각형이 매 프레임 같은 방식으로 그려지는 것은 아닙니다. Nanite는 필요한 지오메트리를 선택하고 공급하는 구조로 그 관계를 바꿉니다.

Nanite는 높은 디테일의 메시를 작은 묶음과 계층으로 준비하고, 화면에 필요한 세부를 선택·공급하는 UE의 가상화 지오메트리 시스템입니다.

고밀도 원본을 항상 같은 삼각형 집합으로 그리는 대신 가시성과 허용 오차에 맞춰 처리합니다. 전통 수동 LOD의 일부 역할을 다른 구조로 수행하지만 재질·조명 전체를 대신하지는 않습니다.

작은 묶음과 화면 오차로 세부를 선택한다

Nanite로 준비한 메시도 카메라와 배치 조건에 따라 필요한 세부가 달라집니다. 선택된 클러스터와 데이터가 전용 가시성·래스터 처리에 사용됩니다.

빌드 과정에서 메시를 삼각형 묶음인 클러스터와 그 계층으로 구성합니다. 여러 세부 수준을 선택할 수 있는 기반을 만듭니다. 런타임에 가시성과 화면 오차를 기준으로 필요한 묶음을 선택합니다. 작게 보이는 부분은 더 거친 표현으로 충분할 수 있으므로 원본 삼각형 수가 그대로 매 프레임 표시 수가 되지 않습니다. 필요한 메시 데이터를 스트리밍으로 공급하고 전용 경로로 화면에 표시합니다. 선택된 형상의 실루엣과 얇은 면의 보존은 실제 출력 조건에서 확인합니다.



직접 제작 설명도 · 실제 Nanite 클러스터 분할이나 측정 개수를 재현한 그림은 아닙니다.

원거리 선택도 비교를 위해 같은 크기로 확대한 개념도입니다. 삼각형을 줄인 경계와 원래 외곽의 차이가 나타나며 실제 Nanite 클러스터 분할의 재현은 아닙니다.

고밀도 바위와 얇은 잎의 조건은 다르다

바위·절벽·건축의 고밀도 디테일을 활용하고 에셋 예산을 판단할 때 필요합니다. 얇은 잎·촘촘한 겹침·정점 변형은 별도로 적용 적합성을 확인합니다.

바위의 촘촘한 표면은 화면 오차 안에서 세부를 줄일 여지가 있지만, 얇은 잎이 겹친 수관에서는 작은 면의 소실과 중첩이 다른 문제를 만듭니다. 고밀도 원본을 쓸 수 있다는 이점과 모든 종류의 형상이 같은 효율을 얻는다는 주장은 구분됩니다.

지오메트리 가상화 뒤에도 남는 비용

인스턴스와 보이는 클러스터·겹침·재질·스트리밍이 비용을 만듭니다. 원본 폴리곤 제한이 완화돼도 조명·그림자와 화면 중복 계산은 남습니다.

Nanite는 폴리곤을 무제한으로 써도 된다는 허가보다 원본과 화면 처리량을 연결하는 다른 방식입니다. 디테일 선택의 원리를 이해하면 원본 개수 밖의 겹침·재질·스트리밍 비용도 함께 보입니다.

참고 자료

[17] Instanced Static Mesh Component

[19] Nanite Virtualized Geometry

18 Culling 보이지 않는 배경을 언제 제외할까

건물을 하나로 합치면 관리하기는 편하지만 모퉁이 너머의 구역까지 함께 후보에 남을 수 있습니다. 렌더러가 어떤 단위로 가시성을 판단하는지에 따라 같은 외관의 배치도 처리량이 달라집니다.

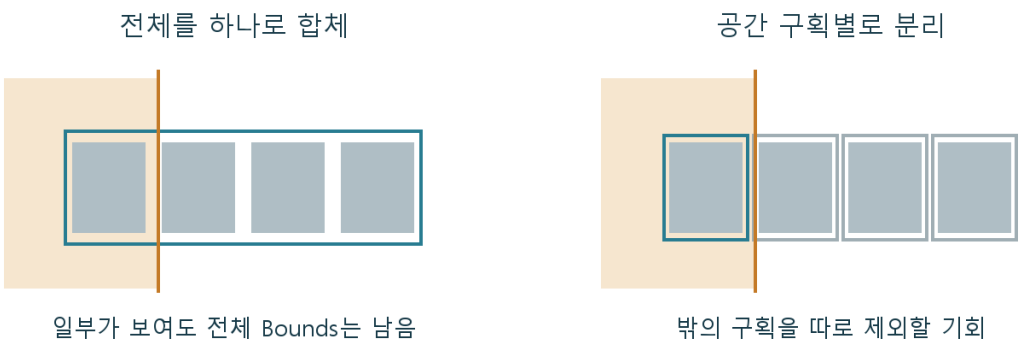
Culling은 화면을 만드는 데 필요하지 않은 렌더링 후보를 제외하는 과정입니다. 거리, 카메라 범위, 다른 물체에 가려졌는지 같은 기준을 사용합니다.

뒤 단계가 처리할 후보를 줄입니다. 모든 물체를 끝까지 계산한 뒤 보이지 않는 색을 버리는 것보다 일찍 제외할 수 있을수록 다른 작업을 줄일 기회가 생깁니다.

시야 밖 제외와 가려짐 판정의 차이

카메라와 물체의 범위, 가림 정보와 설정이 이후 단계에 넘길 후보를 정합니다. 메인 카메라에 불필요한 표면도 광원의 그림자에는 필요할 수 있어 관점에 따라 조건이 다릅니다.

거리 기준과 카메라 시야 공간인 프러스텀 등을 이용해 후보를 줄입니다. 프러스텀 밖 제외와 다른 표면 뒤의 가림 판정은 서로 다른 검사입니다. 물체를 감싸는 상자나 구인 Bounds를 검사 단위로 사용할 수 있습니다. 큰 묶음의 일부가 보이면 전체가 후보에 남을 수 있어 분할 단위가 중요합니다. 깊이 정보 등으로 가려진 후보를 더 줄일 수 있습니다. 실제 검사 순서·지연·세부 단위는 경로에 따라 다르며 Nanite의 클러스터 컬링과 일반 Actor 범위도 구분합니다.



주황 영역은 카메라 범위 안 · 사각 테두리는 Bounds · 같은 건물을 다른 단위로 판정
직접 제작 가시성 개념도 · 실제 원근 시야나 엔진의 모든 컬링 단계를 재현한 그림은 아닙니다.

큰 Bounds와 구획별 Bounds의 후보 판정을 단순화한 도식입니다. 실제 원근 시야와 모든 컬링 단계를 그대로 재현하지는 않습니다.

큰 Bounds가 남겨 두는 불필요한 후보

건물과 실내 모듈의 합체·분할, 모서리에서의 갑작스러운 등장, WPO 물체의 사라짐을 진단할 때 필요합니다. 제작 편의뿐 아니라 카메라가 가리는 공간 단위도 고려합니다.

건물 전체를 감싼 Bounds 중 작은 부분이 보이면 큰 묶음이 가시 후보로 남을 수 있습니다. 공간별로 나뉜 메시에는 뒤 구획을 따로 제외할 기회가 생깁니다. 다만 메인 화면에 안 보이는 표면도 그림자에는 필요할 수 있으므로 카메라 가시성만으로 모든 일을 설명하지는 못합니다.

잘게 나누는 일에도 대가가 있다

분할하면 제외 기회가 늘지만 관리·검사·제출 작업도 늘 수 있습니다. 메인 화면에 안 보이는 물체의 그림자 기여가 남는지도 함께 봅니다.

컬링의 효율은 메시의 총량뿐 아니라 제외할 수 있는 단위에 달려 있습니다. 합체와 분할은 모델링 편의의 선택이면서 동시에 가시성 구조의 선택입니다.

참고 자료

[17] Instanced Static Mesh Component

[19] Nanite Virtualized Geometry

[30] Visibility and Occlusion Culling

19 Mipmap 먼 텍스처는 왜 덜 세밀해야 안정적일까

원본 텍스처가 더 선명해졌는데 멀리 있는 바닥은 오히려 흔들릴 수 있습니다. 작은 화면 픽셀 하나가 많은 무늬를 대표해야 하기 때문입니다. 밉맵은 이 축소 과정에서 필요한 정보를 미리 준비합니다.

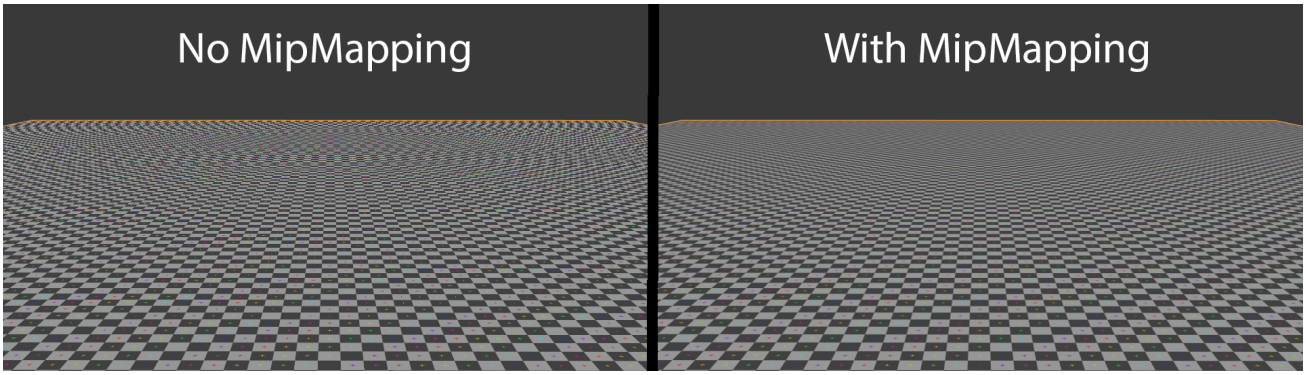
텍스처 샘플링은 UV 등으로 이미지에서 필요한 값을 읽는 작업입니다. Mipmap은 작게 보이는 표면을 안정적으로 읽도록 원본을 단계적으로 축소해 준비한 이미지 계층입니다.

화면 픽셀보다 세밀한 패턴을 적절한 크기로 대표하게 합니다. 원거리 깜빡임·모아레를 줄이고 불필요하게 큰 텍스처 데이터를 읽는 일을 줄이는 데 사용됩니다.

한 픽셀이 많은 텍셀을 대표할 때

텍스처와 UV뿐 아니라 화면에서 UV가 얼마나 빠르게 변하는지도 읽을 정보를 정하는 데 쓰입니다. 적절한 밉과 샘플을 섞은 값이 해당 표면 위치의 재질 계산으로 이어집니다.

원본을 축소해 여러 크기의 밉을 준비합니다. 텍셀은 텍스처 데이터의 한 칸이며, 먼 표면에서는 화면 픽셀 하나가 많은 텍셀을 대표해야 합니다. 화면에서의 UV 변화 등을 이용해 적절한 밉을 선택합니다. 너무 세밀한 원본에서 한 점만 읽으면 작은 이동에도 서로 다른 무늬를 고르기 쉽습니다. 필터링으로 샘플과 밉 사이의 값을 섞습니다. 비스듬한 바닥처럼 한 방향으로 길게 늘어난 범위에는 이방성 필터링이 도움을 줄 수 있습니다.



외부 렌더에서 먼 바닥의 모아레와 패턴 차이가 드러납니다. Blender 결과이며 필터링도 관여할 수 있어 UE의 밉만 바꾼 통제 비교는 아닙니다.

BillyBob CornCob · Wikimedia Commons · 출처 [43]

원본 전체와 비율 유지 · CC0 1.0

이용 조건

정지 선명도와 이동 안정성이 다른 이유

먼 바닥 줄눈과 반복무늬, 미세 노멀의 안정성을 조절할 때 필요합니다. 원본 해상도를 늘리는 것과 플레이 화면에서 읽히는 패턴을 만드는 일은 다릅니다.

촘촘한 흑백 무늬를 원본의 한 점으로만 읽으면 작은 카메라 이동에도 검정과 흰색이 번갈아 선택될 수 있습니다. 적절히 축소한 밉은 개별 선을 덜 보여 주는 대신 넓은 영역의 대표값을 제공합니다. 일부 선명도가 줄어드는 것이 오히려 올바른 표현인 경우입니다.

더 저장하고 더 작은 데이터를 읽는 균형

밉 체인은 저장 데이터를 더 필요로 하지만 작은 밉 사용은 캐시·대역폭에 유리할 수 있습니다. 실제 상주 메모리는 스트리밍과 압축 등에도 달려 있습니다.

밉맵은 텍스처를 막연히 흐리게 만드는 장치가 아닙니다. 화면이 표현할 수 있는 정보량에 맞춰 데이터를 읽게 하는 준비입니다. 패턴의 크기와 필터링까지 연결해야 선명함과 안정성의 균형이 보입니다.

참고 자료

[20] Texture Asset Editor

[21] Texture Streaming Overview

20 Texture Streaming 선명한 원본이 잠시 흐리게 보이는 이유

가까이 다가간 소품이 잠깐 흐렸다가 선명해지는 현상은 원본 해상도가 부족한 경우와 다릅니다. 필요한 데이터와 지금 준비된 데이터 사이에 차이가 있기 때문입니다. 텍스처 스트리밍은 그 차이를 관리합니다.

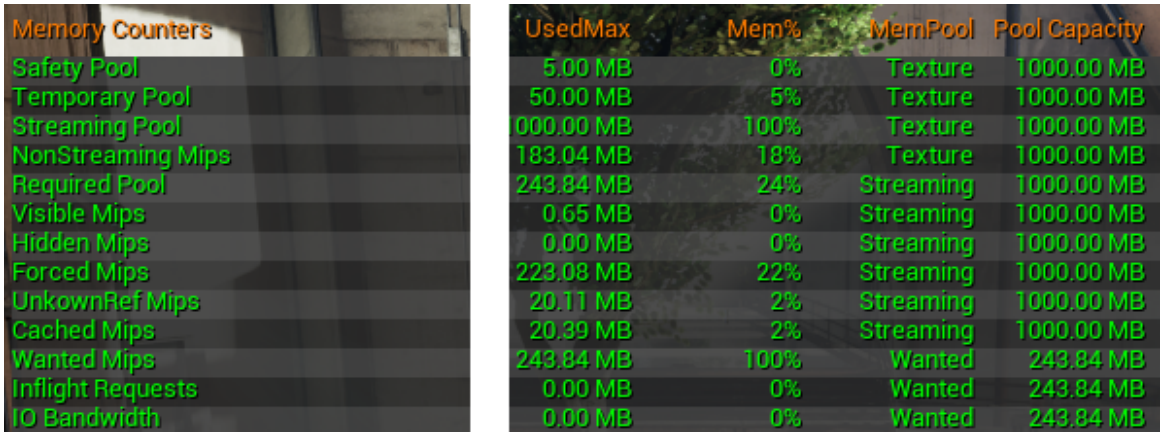
Texture Streaming은 화면에 필요한 텍스처 밉을 추정하고 메모리 예산 안에서 불러오거나 내보내는 관리 시스템입니다. 원본 텍스처를 항상 최고 해상도로 메모리에 두는 방식과 다릅니다.

많은 에셋을 사용하는 장면에서 메모리와 로딩을 관리합니다. CH19의 밉 선택이 읽을 세부 수준의 문제라면, 스트리밍은 필요한 데이터가 실제로 준비돼 있는지의 문제입니다.

원하는 밉과 메모리에 있는 밉 사이

장면의 배치와 화면 요구량이 텍스처 수요를 만들고 메모리 예산이 준비할 범위를 제한합니다. 로드·해제 요청과 실제 상주하는 밉의 상태는 그 관리 결과입니다.

카메라와 물체 크기·거리 등을 이용해 필요한 세부 수준을 추정합니다. 화면에서 원하는 밉과 현재 준비된 밉을 구분합니다. 예산과 우선순위에 맞춰 로드·해제를 결정합니다. 스트리밍 풀은 이 관리에 쓰는 메모리 예산의 한 묶음이며 전체 GPU 메모리와 같은 뜻은 아닙니다. 디스크에서 데이터를 옮겨 요청한 밉을 준비합니다. 이 전송에 시간이 걸리므로 이동 직후 잠시 낮은 밉으로 보이다 선명해질 수 있습니다.



공식 통계 표의 예산·필요량·현재 상태는 서로 다른 항목입니다. 표시된 숫자는 권장 예산이 아니며 UI는 원문의 UE 5.5 기준입니다.

© Epic Games · 공식 문서 · 출처 [54]
원본 통계 표 발췌 · 가운데 여백만 축소 · 값과 항목 유지 · 원저작권과 원문 출처 표시는 유지합니다

고해상도 텍스처를 더 만드는 것이 답이 아닐 때

큰 환경의 텍스처 해상도 배분, 흐린 소품과 팝인, 첫 진입 끊김을 조사할 때 필요합니다. 원본 크기·수요 추정·예산·전송 지연 중 원인을 구분합니다.

빠른 이동으로 높은 밉의 수요가 갑자기 늘면 전송과 예산 조정이 뒤따릅니다. 이미 준비된 구역을 다시 볼 때와 처음 진입할 때의 인상이 달라질 수 있는 이유입니다. 계속 흐린 상태는 최대 해상도나 수요 추정, 예산 조건과도 연결됩니다.

풀의 경고와 전체 메모리 여유

높은 밉의 메모리와 데이터 전송이 부담입니다. 풀을 키우면 다른 리소스 여유가 줄 수 있어 경고를 없애는 것만을 목표로 값을 늘리지 않습니다.

스트리밍을 이해하면 원본의 세밀함과 실행 중 준비 상태를 따로 생각하게 됩니다. 텍스처 해상도 배분은 파일의 크기만 정하는 일이 아니라 필요한 순간의 데이터 수요를 설계하는 일입니다.

참고 자료

[20] Texture Asset Editor

[21] Texture Streaming Overview

[22] Texture Streaming Metrics

21 Lumen 표면 사이를 이동하는 빛을 계산하는 방법

직접 빛이 닿지 않는 흰 벽에도 주변의 색이 묻어납니다. 거울과 매끈한 바닥에는 카메라가 바라보는 표면 밖의 장면도 나타납니다. Lumen은 이렇게 장면 안에서 전달되는 간접광과 반사를 실시간으로 다루는 시스템입니다.

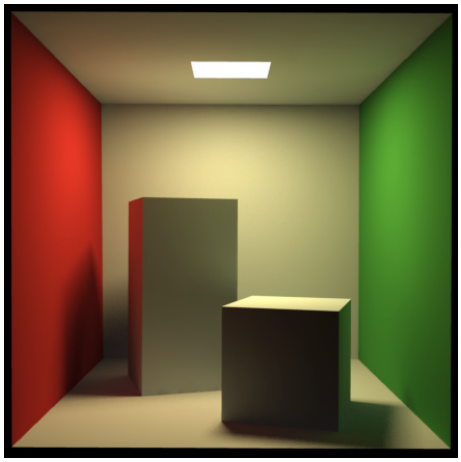
Lumen은 UE에서 동적 전역 조명과 반사를 계산하는 시스템입니다. 전역 조명은 표면 사이에서 전달되는 빛을 포함하는 조명이며, Lumen은 변화하는 장면의 간접광과 반사를 실시간으로 근사합니다.

직접광만으로 설명되지 않는 실내 밝기와 색 번짐, 주변 장면의 반사를 만듭니다. 재질은 같은데 주변 표면이나 광원을 바꾸면 외관이 달라지는 빛 전달을 담당합니다.

화면 정보만으로 부족한 빛의 경로

형상과 재질, 조명과 카메라 정보가 빛 전달의 조건을 만듭니다. Lumen은 추적과 표면 정보 조회로 간접광과 반사를 구해 표면을 조명하는 데 사용합니다.

광원에서 바로 도달한 빛과 다른 표면을 거쳐 온 빛을 구분합니다. 붉은 벽에서 반사된 빛이 흰 벽을 붉게 비추는 것이 간접광의 예입니다. 화면에 있는 정보로 추적을 수행할 수 있고, 화면만으로 부족한 부분은 별도 장면 표현과 지원되는 소프트웨어·하드웨어 추적으로 보완합니다. 화면 밖 물체도 빛 전달에 참여하기 때문입니다. Surface Cache 같은 표면 표현에서 재질·조명 정보를 조회하고 결과를 갱신·통합합니다. 반사의 조명 평가 방식과 추적 설정은 사용 경로와 품질·비용 조건에 따라 선택합니다.



외부 경로 추적 Cornell box 렌더입니다. 색 벽이 주변 표면에 전달하는 간접광을 보여 주며 Lumen 실행 결과는 아닙니다.

Trevorgoodchild · Public domain · 원본 전체·색상·비율 유지 · 출처 [36]

흰 벽의 붉은 기운은 텍스처색이 아닐 수 있다

실내 누광·빛 번짐·반사 누락과 재질색을 구분하고 메시 구성·조명 예산을 정할 때 필요합니다. 흰 벽의 붉은 기운을 바로 텍스처에서 상쇄하기 전에 주변 빛 전달을 봅니다.

붉은 벽이 반사한 빛이 흰 벽에 도달하면 수신 표면의 Base Color를 바꾸지 않아도 최종 색이 달라집니다. 이를 푸른 텍스처색으로 상쇄하면 다른 조명에서는 벽 자체가 틀어질 수 있습니다. 재료의 고유 색과 주변에서 들어오는 빛을 나눠 보는 이유입니다.

움직이는 장면 표현을 유지하는 비용

장면 표현의 갱신, 추적, 조회와 화면 결과의 통합에 비용이 듭니다. 복잡한 장면·변화·품질 설정에 따라 달라지므로 간접광과 반사 관련 작업을 따로 관찰합니다.

Lumen의 역할은 장면을 전체적으로 밝게 만드는 스위치보다 넓습니다. 표면과 표면 사이의 빛 전달을 근사하기 때문에 메시의 구성과 재질, 주변 조명도 결과에 함께 참여합니다.

참고 자료

[23] Lumen Technical Details

[24] Lumen Performance Guide

22 Virtual Shadow Maps 선명한 그림자와 캐시의 관계

정지한 장면에서 한 번 만든 그림자를 다시 쓸 수 있다면 매번 같은 깊이를 그릴 필요가 줄어듭니다. 그러나 바람이 부는 풀과 이동하는 물체 앞에서는 그 그림자가 곧 낡습니다. VSM은 이 해상도와 재사용, 갱신을 페이지 단위로 다룹니다.

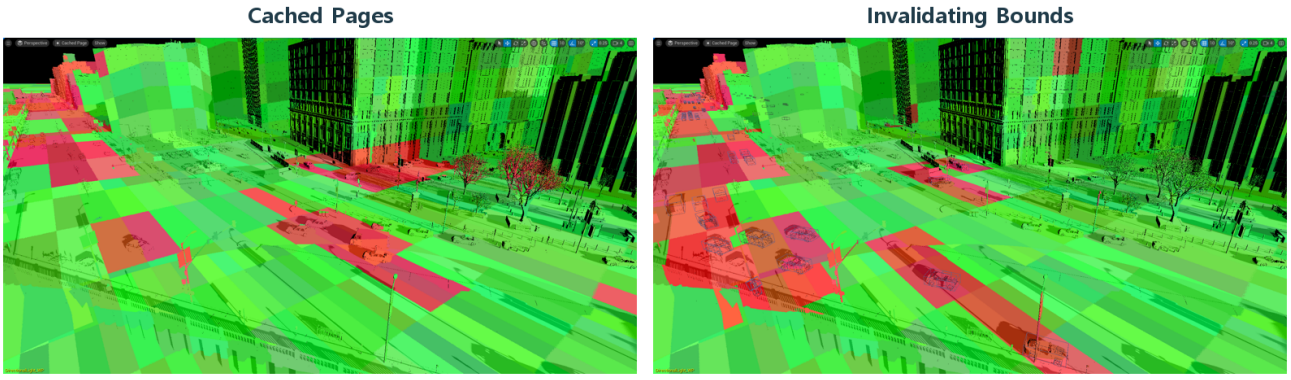
Virtual Shadow Maps는 그림자 데이터를 필요한 작은 페이지 단위로 관리하는 고해상도 그림자 시스템입니다. 광원 관점에서 어떤 표면이 가려지는지 계산하는 그림자 맵의 가상화 방식입니다.

화면에 필요한 그림자 디테일에 데이터를 배분하고, 변하지 않은 결과를 재사용합니다. 해상도와 메모리·갱신 작업을 관리하는 구조이지 그림자를 한 번 그려 영구히 쓰는 기능은 아닙니다.

필요한 그림자 페이지를 준비하고 재사용한다

광원과 보이는 영역, 그림자를 만드는 형상과 변화 정보가 필요한 페이지를 정합니다. 준비한 깊이는 화면 표면의 그림자 여부를 평가하는 데 사용됩니다.

현재 시야와 광원에서 필요한 그림자 페이지를 정합니다. 필요한 작은 영역을 할당·준비하므로 거대한 모든 영역을 항상 같은 해상도로 새로 그리지 않습니다. 기존 페이지가 유효하면 재사용합니다. 카메라 이동으로 새 영역이 필요하거나 광원·형상이 변해 결과가 낡으면 새로 그리거나 무효화 후 갱신합니다. 준비한 깊이로 조명 계산 시 가림을 평가합니다. 깊이를 만드는 일과 화면에 그림자를 적용하는 일은 서로 다른 작업으로 관찰할 수 있습니다.



공식 VSM 진단에서 재사용 페이지와 새로 필요하거나 무효화된 페이지가 구분됩니다. 오른쪽에는 갱신에 영향을 주는 Bounds도 표시돼 있습니다.

© Epic Games · 공식 문서 · 출처 [56]
동일 범위의 원본을 나란히 배치 · 표제 추가 · 원저작권과 원문 출처 표시는 유지합니다

카메라가 멈춰도 풀밭의 그림자가 갱신되는 이유

식생의 바람·이동 소품·광원 변화가 그림자 비용에 미치는 영향을 이해할 때 필요합니다. WPO와 큰 Bounds가 어느 범위의 갱신을 유발하는지 조사합니다.

WPO로 잎의 위치가 달라지면 이전 그림자는 현재 형상과 맞지 않습니다. 영향 영역의 페이지가 무효화되고 다시 그려질 수 있습니다. 카메라가 새 영역을 보아서 만든 페이지와 움직임 때문에 갱신한 페이지는 겉으로 비슷해도 원인이 다릅니다.

바람의 외관과 캐시 재사용의 균형

필요 페이지의 메모리, 새 페이지·무효화 영역의 렌더링, 그림자 평가가 비용입니다. 정지 장면의 캐시 효과와 움직이는 장면의 갱신 비용을 구분합니다.

VSM을 이해하면 그림자 품질만 낮추는 대신 어떤 변화가 얼마나 넓은 갱신을 만드는지 생각할 수 있습니다. 선명함과 움직임, 페이지 재사용은 함께 설계할 조건입니다.

참고 자료

[19] Nanite Virtualized Geometry

[25] Virtual Shadow Maps

23 TAA 과거 프레임이 경계를 안정시키고 잔상을 만드는 이유

철망의 깜빡임은 줄었는데 이동 중에는 흔적이 남을 수 있습니다. 서로 반대처럼 보이는 두 결과가 이전 프레임을 활용하는 같은 구조에서 나옵니다. TAA는 현재 화면만으로 부족한 정보를 시간에 걸쳐 보완합니다.

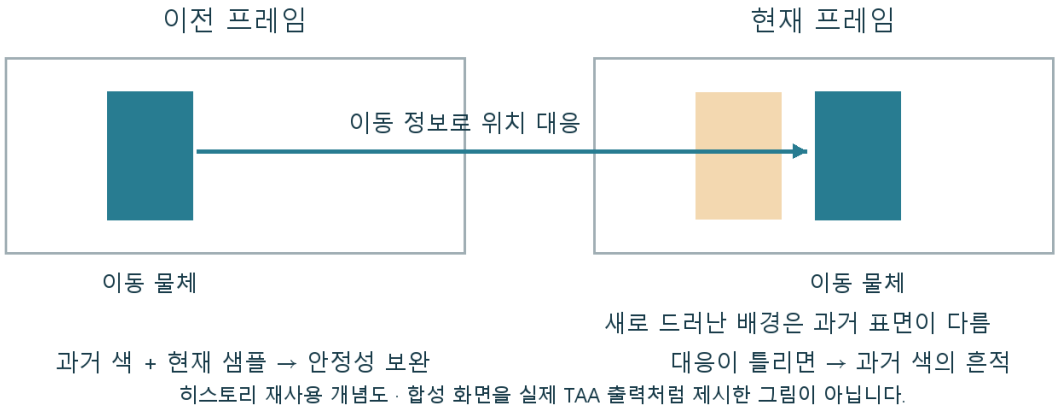
TAA는 Temporal Anti-Aliasing으로, 현재와 이전 프레임의 정보를 이용해 경계의 계단 현상과 시간적 깜빡임을 줄이는 안티앨리어싱 방식입니다.

제한된 픽셀 샘플이 놓친 세부를 여러 프레임에 걸쳐 보완합니다. 단일 정지 이미지의 경계뿐 아니라 카메라 이동 중 얇은 패턴의 안정성을 담당합니다.

과거의 표면을 현재 위치에 대응시킨다

현재 색만으로는 누적할 수 없습니다. 이전 결과인 히스토리와 깊이, 이동 정보 등을 이용해 과거 표면을 현재 위치에 대응시키고 신뢰할 수 있는 정보를 결합합니다.

프레임마다 조금 다른 샘플 위치로 장면을 계산합니다. 같은 얇은 선도 여러 위치에서 검사하면서 한 프레임이 얻지 못한 정보를 모을 수 있습니다. 물체·카메라의 이동 정보로 이전 위치와 현재 위치를 대응시킵니다. Velocity는 화면상의 이동 정보이며, 변형 재질에도 올바른 대응이 필요합니다. 맞는 히스토리는 현재 정보와 결합하고 불확실한 과거 정보는 줄이거나 버립니다. 새로 드러난 표면과 잘못된 대응에서는 잔상·불안정성이 나타날 수 있습니다.



히스토리 재사용 개념도 · 합성 화면을 실제 TAA 출력처럼 제시한 그림이 아닙니다.

이전 물체 위치를 현재에 대응시키고, 새로 드러난 배경에는 다른 과거 정보가 있음을 보여 주는 직접 제작 시간 흐름도입니다.

새로 드러난 배경에는 같은 과거가 없다

철망·가는 가지·WPO 식생과 이동하는 재질의 깜빡임·잔상을 진단할 때 필요합니다. 패턴 크기와 이동 정보를 관리하고 정지·이동·가림 해제를 구분해 관찰합니다.

움직이는 물체 뒤에서 배경이 나타나면 그 위치의 과거 화면에는 다른 표면이 들어 있었습니다. 잘못 대응한 정보를 오래 유지하면 흔적이 남고, 과거 정보를 적게 쓰면 안정성의 이득도 줄 수 있습니다. 정지한 한 장으로 이동 품질을 설명하기 어려운 이유입니다.

시간적 안정성과 정보의 신뢰도

현재·과거 버퍼를 읽고 대응·필터링하는 화면 계산과 저장 비용이 있습니다. 화질은 히스토리 조건에도 달려 있어 정지 캡처 한 장으로 품질을 확정하지 않습니다.

TAA의 화질은 경계를 얼마나 매끈하게 만들었는지만으로 정해지지 않습니다. 과거 정보를 언제 믿고 언제 버리는지가 중요하며, 아트의 미세 패턴과 변형 정보도 그 판단 조건을 만듭니다.

참고 자료

[26] Anti-Aliasing and Upscaling

[27] Temporal Super Resolution

24 TSR 내부 해상도와 출력 해상도 사이의 재구성

화면에 표시되는 4K 이미지가 처음부터 모든 작업을 4K로 계산한 결과일 필요는 없습니다. TSR은 현재의 입력과 과거 프레임을 이용해 목표 출력 이미지를 재구성합니다. 표시 크기와 실제 입력 정보량을 구분해야 하는 이유입니다.

TSR은 Temporal Super Resolution으로, 현재 프레임과 히스토리를 활용해 목표 출력 해상도의 이미지를 재구성하는 UE의 시간적 업스케일러입니다.

낮은 내부 해상도로 장면을 계산하면서 높은 출력 해상도를 제공할 수 있게 합니다. 경계를 안정시키는 시간적 누적을 해상도 재구성과 연결하는 시스템입니다.

적은 입력 픽셀을 시간적 정보로 보완한다

내부 해상도의 장면 색과 이동·깊이 정보, 히스토리가 목표 출력 이미지를 만드는 재료가 됩니다. 계산을 시작한 픽셀 수와 표시할 픽셀 수가 같지 않을 수 있습니다.

Screen Percentage 등으로 내부 렌더링 해상도를 정합니다. 가로·세로 비율을 함께 낮추면 장면을 계산할 픽셀 수가 더 크게 줄어듭니다. 이동 정보를 이용해 과거 세부를 현재 위치에 대응시키고 현재 샘플과 결합합니다. 단순히 한 장을 늘리는 것보다 시간적으로 얻은 정보를 활용합니다. 신뢰할 수 없는 히스토리를 처리하며 목표 출력으로 재구성합니다. 새로 드러나는 글자·얇은 선·빠른 반사는 입력 정보의 부족을 드러내기 쉽습니다.

Native 4K



1080p → 4K TSR



동일 영역을 확대한 공식 TSR 비교입니다. 네이티브 4K와 1080p에서 4K로 재구성한 결과이며 정지 이미지가 이동 품질까지 보장하지는 않습니다.

© Epic Games · 공식 문서 · 출처 [58]
원본 일부 발췌·확대 · 색상 유지 · 표제 추가 · 원저작권과 원문 출처 표시는 유지합니다

같은 출력 크기에서도 이동 품질이 다른 이유

목표 기기의 화면 품질과 GPU 예산을 맞출 때 필요합니다. 출력 해상도만 보고 같은 화질이라고 결정하지 말고 내부 해상도와 이동 결과를 함께 봅니다.

입력이 가로·세로 67%라면 픽셀 수는 약 45%입니다. 하지만 새로 드러난 글자나 빠른 반사에는 활용할 과거 정보가 부족할 수 있습니다. 정지 상태에서는 비슷해 보여도 이동과 가림 해제에서 차이가 드러날 수 있습니다.

픽셀 감소율이 프레임 시간 감소율은 아니다

장면의 화면 단위 작업을 줄일 수 있지만 TSR 자체의 처리·저장 비용과 고정 비용은 남습니다. 입력 픽셀 감소율이 총 프레임 시간 감소율은 아닙니다.

TSR은 낮은 해상도를 그대로 늘리는 효과보다 장면 계산의 예산을 배분하는 재구성 시스템입니다. 높은 출력 해상도의 이름보다 입력 조건과 이동 화질, 전체 GPU 시간이 더 실질적인 기준입니다.

참고 자료

[26] Anti-Aliasing and Upscaling

[27] Temporal Super Resolution

25 CPU와 GPU 병목 가벼워진 에셋이 FPS를 올리지 못할 때

폴리곤과 해상도를 줄였는데 FPS가 그대로인 일은 이상한 예외가 아닙니다. 줄인 일이 프레임을 제한하던 원인이 아니었다면 전체 진행 속도는 달라지지 않을 수 있습니다. 병목 분석은 이 제한 관계를 찾는 일입니다.

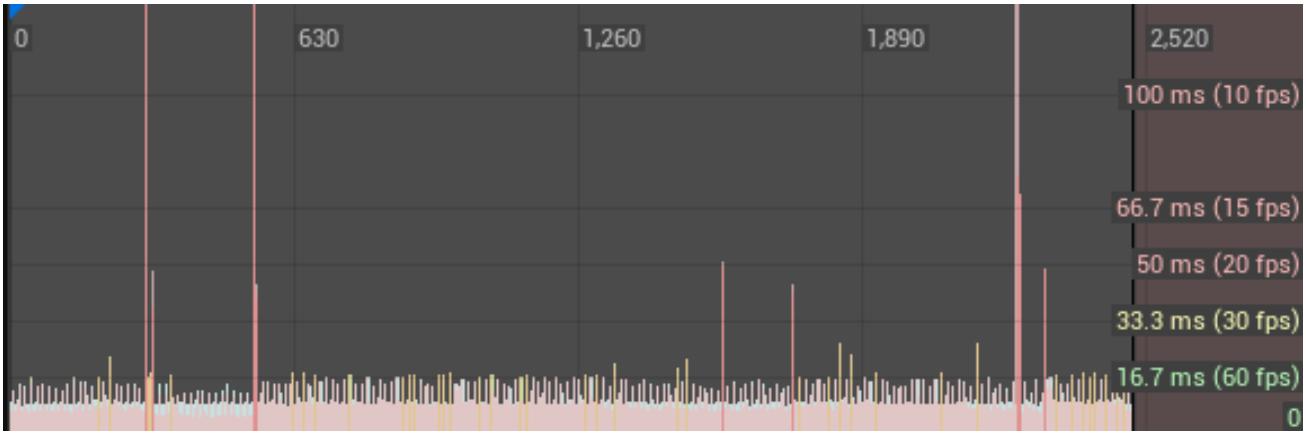
병목은 프레임의 처리율이나 완료 시간을 제한하는 작업 또는 대기 관계입니다. CPU와 GPU 병목 분석은 어느 쪽의 준비·실행이 실제로 프레임 진행을 제한하는지 찾는 과정입니다.

효과가 있는 최적화 대상의 우선순위를 정합니다. 눈에 복잡한 에셋을 줄이는 것과 프레임을 제한하는 일을 줄이는 것은 같지 않으므로 먼저 제한 원인을 확인합니다.

준비와 실행이 겹치는 프레임의 구조

CPU·GPU·프레임 시간과 타임라인은 같은 구간의 실행이 어디에서 제한되는지 보여 주는 단서입니다. 계산과 제출, 동기화와 로딩을 나눠 이해해야 어떤 변경이 의미 있을지 판단할 수 있습니다.

CPU의 게임 상태 갱신과 명령 준비, GPU의 실행은 여러 프레임에서 겹칠 수 있습니다. 단순히 모든 시간을 더해서 프레임 시간을 구하지 않습니다. stat unit의 Frame·Game·Draw·GPU는 비용의 큰 방향을 보여 줍니다. 프레임 제한과 VSync에 의한 대기도 이 숫자의 해석에 영향을 주므로, 긴 값 하나가 곧 수정할 계산이라는 뜻은 아닙니다. 해상도처럼 영향 범위가 알려진 조건에 대한 시간 변화는 제한 원인의 단서가 됩니다. 타임라인은 계산과 기다림, 작업의 의존 관계를 드러내므로 같은 FPS에서도 서로 다른 원인을 구분할 수 있습니다.



외부 Unreal Insights의 프레임 시간 그래프입니다. 기준선과 시간 변화가 드러나지만 CPU·GPU·대기의 원인은 개별 시간축과 함께 읽어야 합니다.

© John Farrow · Unrealcode.net · 출처 [59]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 원저작권과 원문 출처 표시는 유지합니다

긴 막대가 모두 계산을 뜻하지는 않는다

폴리곤과 재질을 줄여도 FPS가 그대로일 때, 첫 진입만 끊길 때, CPU 제출이 무거운 대규모 배치에서 필요합니다. 수정할 에셋인지 실행·관리 구조인지 조사 방향을 정합니다.

CPU가 다음 명령을 준비하느라 늦는 동안 GPU 작업만 줄이면 GPU의 여유는 늘어도 전체 프레임은 크게 달라지지 않을 수 있습니다. 반대로 프레임 제한과 VSync 때문에 기다리는 상태도 있습니다. 시간축에서는 실제 계산과 기다림, 서로 의존하는 구간을 구분해야 합니다.

부분 최적화와 전체 개선의 차이

GPU만 가벼워져도 CPU가 제한하면 총 프레임은 거의 그대로일 수 있습니다. 가장 긴 막대도 대기를 포함할 수 있어 실제로 제한하는 관계를 확인해야 합니다.

병목을 안다는 것은 가장 복잡해 보이는 물체를 고르는 것이 아닙니다. 프레임의 진행을 실제로 제한하는 일을 찾는 것입니다. 그 관계를 이해해야 에셋 수정이 필요한 상황과 다른 실행 구조를 봐야 할 상황이 갈립니다.

참고 자료

[03] Introduction to Performance Profiling and Configuration

[28] Stat Commands

26 GPU Profiler 프레임의 비용을 읽는 법

프레임이 느리다는 사실과 무엇을 고쳐야 하는지는 다른 정보입니다. GPU Profiler는 표면·그림자·반투명·후처리 같은 작업의 시간을 드러내지만, 그 비용을 만드는 제작 원인까지 자동으로 결정해 주지는 않습니다.

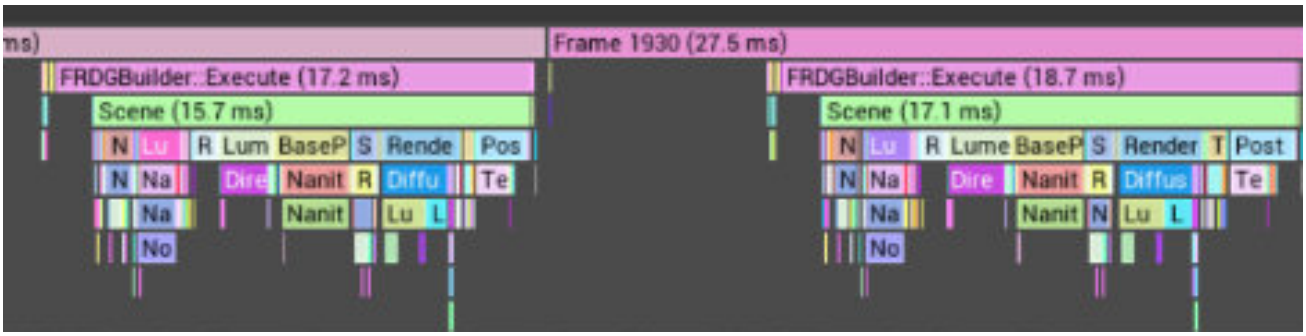
GPU Profiler는 GPU 작업을 이벤트와 렌더 패스별 시간으로 관찰하는 도구입니다. 프로파일링은 실행 중 어디에 시간이 쓰이는지 측정하고 원인 후보를 좁히는 과정입니다.

느린 프레임을 구성하는 작업을 찾아 아트 변경의 가설과 연결합니다. 도구는 비용의 위치를 보여 주며, 어떤 제작 선택이 원인인지는 비교 실험으로 검증합니다.

패스와 이벤트로 드러나는 GPU의 일

GPU 캡처의 이벤트와 시간 정보는 패스의 계층과 시간축으로 드러납니다. 다른 조건의 실행과 비교할 때도 각 구간이 무엇을 포함하는지 알아야 숫자를 같은 의미로 읽을 수 있습니다.

stat gpu는 큰 GPU 작업 묶음을, 지원되는 profilegpu와 Unreal Insights는 더 세부적인 시간과 이벤트를 보여 줍니다. 그림자·표면·반투명·후처리가 프레임 안에서 차지하는 위치를 읽는 정보입니다. 부모 이벤트와 세부 구간을 펼쳐 어느 작업이 시간을 쓰는지 찾습니다. 포함 시간과 자체 시간을 구분하고 겹쳐 실행된 작업을 단순 합산하지 않습니다. 이벤트의 비용은 그것을 만드는 에셋과 기능에 대한 설명으로 이어져야 합니다. 같은 조건의 반복 결과와 해당 패스·전체 프레임·외관의 관계가 있어야 일시적인 숫자 변화와 유효한 개선을 구분할 수 있습니다.



원문 UE 5.6 사례의 외부 GPU Insights 이벤트 계층입니다. 부모 구간 안에 자식이 포함되므로 모든 시간을 단순히 더하면 중복할 수 있습니다.

Tom Looman 문서 수록 · Unreal Engine 도구 화면 · 출처 [60]

원본 일부 발췌·확대 · 색상 유지 · 표제 추가 · 원저작권과 원문 출처 표시는 유지합니다

그림자 시간이 길다는 말만으로 풀을 지울 수는 없다

풀의 그림자 갱신, 큰 반투명, 복잡한 머티리얼 등 제작 변경의 채택 여부를 정할 때 필요합니다. 빠른 한 번의 숫자보다 재현성과 필요한 품질을 유지했는지가 중요합니다.

풀의 변형이 그림자 페이지 갱신을 늘린다면 그림자 구간의 비용과 연결됩니다. 그렇더라도 움직임을 없애는 변경이 제품에 맞는지는 별개입니다. 비용의 위치를 찾는 정보와, 필요한 외관을 유지하며 바꿀 대상을 정하는 판단 사이에는 원인에 대한 설명이 필요합니다.

숫자의 감소가 개선이 되는 조건

캡처·셰이더 준비·로딩 순간은 측정을 흔들 수 있습니다. 안정 구간을 쓰고 대상 기기의 실제 프레임을 확인합니다. 도구의 표시 방식은 UE 버전에 따라 달라집니다.

프로파일러는 어떤 일이 무거운지 보여 주는 지도입니다. 제작의 결정은 그 일이 생기는 이유와 필요한 화면의 가치를 함께 읽을 때 가능해집니다. 패스 하나의 감소보다 실제 목표를 만족하는 변화가 중요합니다.

참고 자료

[03] Introduction to Performance Profiling and Configuration

[28] Stat Commands

[29] Beyond Print String · GPU Profiler

용어 찾아보기

본문의 개념을 다시 찾기 위한 색인입니다. 괄호는 관련 포스트 번호입니다.

CPU와 GPU

CPU는 게임 상태와 작업 준비 등을 처리하는 범용 프로세서, GPU는 많은 그래픽 계산을 병렬로 처리하는 프로세서입니다. [01·25]

스레드

프로그램의 명령을 이어 실행하는 흐름입니다. 렌더 스레드가 CPU에서 일한다는 점과 GPU의 실행을 구분합니다. [01·25]

렌더 스레드

주로 CPU에서 장면의 가시성과 그릴 명령을 준비하는 실행 흐름입니다. [01·25]

RHI와 그래픽 API

RHI는 엔진의 그래픽 명령을 플랫폼별 그래픽 API에 연결하는 계층입니다. API는 기능을 사용하기 위해 정한 요청 방식입니다. [01·25]

프레임과 렌더 패스

프레임은 화면 한 장을 만드는 처리 구간이며, 패스는 그 안에서 특정 입력과 결과를 다루는 작업 단위입니다. [01·26]

지오메트리와 메시

지오메트리는 형태와 위치에 관한 기하 정보입니다. 이 문서의 메시는 주로 삼각형으로 구성한 3D 표면 형상입니다. [01·17]

정점

삼각형의 꼭짓점입니다. 위치 외에 표면 방향과 텍스처 좌표 같은 정보도 가질 수 있습니다. [01·10]

좌표 변환과 투영

모델 내부의 위치를 장면·카메라 기준으로 바꾸고 화면 위치에 대응시키는 과정입니다. [01]

셰이더

GPU에서 실행되는 계산 프로그램입니다. 정점과 화면의 표면 값 등을 처리하는 여러 종류가 있습니다. [01·10]

래스터화

화면에 투영한 삼각형이 어느 화면 위치를 덮는지 판정하는 과정입니다. [01]

용어 찾아보기 계속

본문의 개념을 다시 찾기 위한 색인입니다. 괄호는 관련 포스트 번호입니다.

샘플과 프래그먼트

래스터 문맥의 샘플은 커버리지를 검사하는 화면 위치입니다. 프래그먼트는 삼각형에서 생긴 화면 위치별 처리 후보입니다. [01·05]

보간

알고 있는 값 사이에서 중간 위치의 값을 계산하는 것입니다. 삼각형 내부의 UV 등을 구하는 데 쓰입니다. [01]

UV와 텍스처 샘플

UV는 표면을 이미지의 위치에 대응시키는 2D 좌표입니다. 텍스처 샘플은 그 좌표에서 이미지 값을 읽는 작업입니다. [01·19]

법선과 노멀 맵

법선은 표면 방향입니다. 노멀 맵은 작은 표면 방향 변화를 이미지로 표현해 빛 반응에 사용합니다. [03·07]

탄젠트

표면을 따라 놓인 방향 기준입니다. 탄젠트 공간 노멀 맵은 이 로컬 방향 기준을 이용해 표면 방향 변화를 표현합니다. [03]

버퍼와 렌더 타깃

버퍼는 계산 데이터를 보관하는 메모리 공간입니다. 렌더 타깃은 그리기 결과를 기록할 이미지 자원입니다. [01·03]

GBuffer와 Base Pass

GBuffer는 조명에 필요한 표면 정보 묶음입니다. Base Pass는 주로 불투명 표면 정보를 준비하는 작업입니다. [03]

DBuffer와 데칼

데칼은 다른 표면에 투영하는 얼룩·표식 등의 표현입니다. DBuffer는 Base Pass 전에 그 표면 수정 정보를 준비합니다. [04]

깊이 버퍼와 Early Z

깊이 버퍼는 앞쪽 표면 깊이를 저장합니다. Early Z는 가능한 경우 이 정보로 가려진 후보를 일찍 제외합니다. [05]

Opaque와 Masked와 Translucent

각각 불투명, 위치별로 남길지 버리는 구멍 마스크, 뒤의 색과 합성하는 반투명 표면을 가리킵니다. [06·12]

용어 찾아보기 계속

본문의 개념을 다시 찾기 위한 색인입니다. 괄호는 관련 포스트 번호입니다.

Overdraw와 커버리지

Overdraw는 같은 화면 위치의 반복 처리입니다. 커버리지는 도형이 화면의 어느 범위를 덮는지를 뜻합니다. [06]

PBR

재료와 빛의 상호작용을 물리적으로 그럴듯하게 근사하는 제작 방식입니다. [07]

Base Color와 Metallic과 Roughness

재질의 기본 색, 금속성, 반사가 퍼지는 정도를 제어하는 입력입니다. 최종 화면 색과 역할이 다릅니다. [07]

Material Instance와 파라미터

인스턴스는 원본 재질 구조를 공유하는 설정값 변형입니다. 파라미터는 바꿀 수 있도록 노출한 입력값입니다. [07·10]

BRDF와 BSDF

BRDF는 방향에 따른 반사 반응을 기술합니다. BSDF는 표면을 통한 투과까지 포괄하는 더 넓은 개념입니다. [08]

Fresnel과 F0

Fresnel은 각도에 따른 계면 반사율의 변화입니다. F0는 법선 방향 입사의 반사율입니다. [09]

컴파일과 셰이더 순열

컴파일은 실행할 프로그램을 준비하는 과정입니다. 순열은 기능·정적 설정 등의 조합에 따른 실행 버전을 뜻합니다. [10]

Static Switch와 WPO

Static Switch는 컴파일할 때 경로를 고릅니다. WPO는 정점의 월드 위치에 변화를 더하는 재질 입력입니다. [10]

캐시와 대역폭

캐시는 데이터를 다시 읽기 쉽게 두는 임시 저장 영역입니다. 대역폭은 일정 시간에 옮길 수 있는 데이터 양입니다. [03·10]

Slab과 Closure

Slab은 Substrate의 재료 정의 단위입니다. Closure는 실행 시 조명에 대해 평가할 재료 반응의 단위를 가리킵니다. [11]

용어 찾아보기 계속

본문의 개념을 다시 찾기 위한 색인입니다. 괄호는 관련 포스트 번호입니다.

파라미터 블렌딩

복수 재료의 설정값을 하나의 재료 반응으로 합쳐 근사하는 방식입니다. [11]

알파 합성과 굴절

알파 합성은 앞뒤 색을 비중에 따라 섞는 계산입니다. 굴절은 물질 경계에서 빛 방향이 바뀌는 현상입니다. [12]

SSS와 MFP

SSS는 표면 아래 빛의 산란입니다. MFP는 내부 상호작용 사이의 평균 거리 스케일을 가리킵니다. [13]

Actor와 컴포넌트

Actor는 레벨에 배치하는 오브젝트 단위입니다. 컴포넌트는 그 안의 메시·조명 등 기능 요소입니다. [14·15]

Draw Call과 메시 섹션

Draw Call은 GPU에 그리기를 제출하는 명령입니다. 섹션은 보통 재질을 따로 적용할 수 있는 메시의 구역입니다. [14]

배칭과 인스턴싱

배칭은 호환되는 작업을 묶는 접근입니다. 인스턴싱은 메시 등의 데이터를 공유해 반복 배치를 처리하는 방식입니다. [14·15]

ISM과 HISM

반복 Static Mesh를 컴포넌트 안에서 관리하는 기능입니다. HISM은 공간적으로 묶은 계층을 활용합니다. [15]

LOD와 FOV

LOD는 필요한 화면 디테일에 맞춘 대체 표현입니다. FOV는 카메라가 담는 시야각입니다. [16]

클러스터와 폴백 메시

Nanite 클러스터는 작은 삼각형 묶음입니다. 폴백 메시는 다른 기능이나 조건에서 사용하는 대체 표현입니다. [17]

컬링과 bounds

컬링은 불필요한 가시 후보를 제외하는 과정입니다. bounds는 이런 판정 등에 쓰는 물체를 둘러싼 범위입니다. [18]

용어 찾아보기 계속

본문의 개념을 다시 찾기 위한 색인입니다. 괄호는 관련 포스트 번호입니다.

밉맵과 필터링

밉맵은 축소 이미지의 계층입니다. 필터링은 필요한 범위의 값을 읽고 섞어 화면에 대응시키는 처리입니다. [19]

스트리밍 풀과 I/O

풀은 관리할 메모리 예산의 한 묶음입니다. I/O는 디스크 등에서 데이터를 읽고 옮기는 입출력 작업입니다. [20]

Screen Trace와 거리장

Screen Trace는 화면 정보로 경로를 조사합니다. 거리장은 표면까지의 거리를 저장한 장면 표현입니다. [21]

Surface Cache와 Cards

Surface Cache는 표면 정보를 빠르게 조회하는 체계입니다. Cards는 표면을 여러 방향에서 담는 표현에 사용됩니다. [21]

레이 트레이싱과 가속 구조

광선과 물체의 교차를 추적하는 계산과, 교차 후보를 빨리 찾도록 준비한 자료 구조입니다. [21]

VSM 페이지와 캐스터

페이지는 가상 그림자 맵의 작은 영역입니다. 캐스터는 그림자를 만드는 물체입니다. [22]

캐시 무효화

예전에 계산해 둔 결과가 현재 장면에 맞지 않아 다시 계산해야 하는 상태입니다. [22]

히스토리와 지터

히스토리는 이전 프레임의 저장 정보입니다. 지터는 프레임마다 샘플 위치를 조금씩 바꾸는 동작입니다. [23]

모션 벡터와 재투영

모션 벡터는 화면상 이동 정보입니다. 재투영은 과거 정보를 현재 화면 위치에 대응시키는 과정입니다. [23]

고스팅

과거 정보가 잘못 남아 움직이는 표면 뒤로 끌리거나 번져 보이는 시간적 아티팩트입니다. [23]

용어 찾아보기 계속

본문의 개념을 다시 찾기 위한 색인입니다. 괄호는 관련 포스트 번호입니다.

내부 해상도와 업스케일러

내부 해상도는 주요 장면 계산의 크기입니다. 업스케일러는 그 결과 등을 이용해 더 큰 출력 화면을 만드는 기능입니다. [24]

병목과 VSync

병목은 처리율을 제한하는 원인입니다. VSync는 디스플레이 갱신에 출력 시점을 맞추는 기능입니다. [25]

중앙값과 P95

중앙값은 크기순 가운데 값입니다. P95는 측정값의 95%가 그 이하에 놓이는 경계로, 느린 프레임을 살필 때 사용합니다. [25·26]

프로파일링과 워밍업

프로파일링은 실행 중의 비용 측정입니다. 워밍업은 셰이더 준비·로딩 등을 끝내 안정된 비교 조건을 만드는 과정입니다. [26]

톤매핑과 Bloom

톤매핑은 장면 밝기를 표시할 범위로 옮기는 처리입니다. Bloom은 밝은 부분이 주변으로 번지는 화면 효과입니다. [01]

Scalability와 쿠킹

Scalability는 엔진의 품질 설정 묶음입니다. 쿠킹은 대상 플랫폼에서 실행할 배포용 데이터를 준비하는 과정입니다. [26]

기술 메모

1 스레드와 파이프라인 도식의 범위

RHI는 인터페이스 계층의 이름입니다. RHI 작업이 별도 스레드에서 처리되는 방식과 병렬화 구성은 플랫폼·설정에 따라 달라집니다. 본문의 단계 그림은 작업 의존 관계를 이해하기 위한 것입니다. 실제 GPU 프레임은 그림자·추적·후처리 등의 분기와 동시 실행을 포함하므로 GPU 캡처에서 확인합니다.

2 BRDF 수식을 더 읽을 때

대표적인 미세면 스펙큘러 항은 D×F×G를 4(N·L)(N·V)로 나눈 형태로 설명할 수 있습니다. D는 미세면 방향 분포, F는 Fresnel 반사율, G는 미세면 사이의 가림입니다. N은 표면 법선, L은 빛 방향, V는 시선 방향이며 점 기호는 두 방향의 관계를 계산하는 내적입니다. 실제 UE 재질의 모든 경로가 이 한 식과 같은 구현이라는 뜻은 아닙니다.

3 Substrate 저장 형식과 버전

본문은 UE 5.8 문서와 참고 프로젝트의 5.8.4를 기준으로 합니다. Adaptive GBuffer의 지원 플랫폼과 단순화 조건, DBuffer 요구 사항은 해당 버전의 공식 문서에서 확인합니다. 60Hz 지향은 설계 목표이며 실제 게임의 60 FPS 달성을 보장하는 수치가 아닙니다. 프로젝트 범위 변경은 재시작이나 셰이더 재컴파일을 요구할 수 있습니다.

4 GPU 프로파일러의 표시와 시간 합계

UE 5.6 이후 구성에서는 구형 GPU Visualizer 팝업과 같은 표시를 전제로 하지 않고 로그와 Unreal Insights의 GPU 트랙을 확인합니다. 중첩 이벤트, 동시 실행과 CPU·GPU의 대기 때문에 항목의 단순 합이 총 프레임과 다를 수 있습니다. 공식 도구 설명과 대상 빌드의 실제 동작을 함께 확인합니다.

5 셰이더 준비와 실행의 시간

셰이더 순열은 설정 조합에 따른 프로그램 변형입니다. 실행 중의 연산 비용과 컴파일 대기, 쿠킹·배포 준비 시간은 서로 다른 지표입니다. 정점 변형의 외관만으로 이 시간을 함께 설명할 수는 없습니다.

출처 목록

각 포스트의 공식 자료와 외부 이미지 원문입니다.

01 Rendering Settings

02 Render Dependency Graph

03 Introduction to Performance Profiling and Configuration

04 Forward Shading Renderer

05 Programming with Substrate GBuffer Formats

06 Decal Materials

07 Viewport Modes

08 Physically Based Materials

09 Using Fresnel

10 Overview of Substrate Materials

11 Material Editor UI

12 Material Inputs

13 Lit Translucency

14 Using Refraction

15 Subsurface Profile Shading Model

16 Mesh Drawing Pipeline

17 Instanced Static Mesh Component

18 Optimizing LOD Screen Size Per Platform

출처 목록 계속

각 포스트의 공식 자료와 외부 이미지 원문입니다.

19 Nanite Virtualized Geometry

20 Texture Asset Editor

21 Texture Streaming Overview

22 Texture Streaming Metrics

23 Lumen Technical Details

24 Lumen Performance Guide

25 Virtual Shadow Maps

26 Anti-Aliasing and Upscaling

27 Temporal Super Resolution

28 Stat Commands

29 Beyond Print String · GPU Profiler

30 Visibility and Occlusion Culling

31 Real Shading in Unreal Engine 4 · Brian Karis

32 Scripting the Editor Using Python

33 Parallel Rendering Overview

34 Direct3D Rasterizer Stage

35 Material Instances

36 Cornellbox pathtracing · Trevorgoodchild · Public domain

출처 목록 계속

각 포스트의 공식 자료와 외부 이미지 원문입니다.

37 LearnOpenGL Coordinate Systems · 이미지 출처

38 LearnOpenGL Anti Aliasing · 이미지 출처

39 LearnOpenGL Deferred Shading · 이미지 출처

40 LearnOpenGL PBR Lighting · 이미지 출처

41 LearnOpenGL PBR Theory · 이미지 출처

42 LOD Example · Sirotk · CC BY-SA 3.0

43 Mipmap Aliasing Comparison · BillyBob CornCob · CC0

44 LearnOpenGL 이미지 이용 조건 · CC BY 4.0

45 Epic Nanite Preserve Area 비교 이미지 · UE 5.5

46 Decal Materials 비교 이미지 · UE 5.5

47 LearnOpenGL Depth Testing 비교 이미지

48 Android Developers Materials and Shaders

49 Substrate 코팅 비교 이미지 · UE 5.4

50 Subsurface 비교 이미지 · UE 5.5

51 Mesh Drawing Pipeline 도식 · UE 5.5

52 LearnOpenGL Instancing 렌더 예시

53 Visibility and Occlusion Culling 단계별 그림

54 Texture Streaming Metrics 통계 화면 · UE 5.5

출처 목록 계속

각 포스트의 공식 자료와 외부 이미지 원문입니다.

55 Lumen 반사 조명 비교 이미지 · UE 5.5

56 VSM 캐시와 무효화 범위 · UE 5.4

57 Anti Aliasing 전후 이미지 · UE 5.5

58 TSR 동일 출력 비교 · UE 5.5

59 Unreal Insights 실측 화면 · John Farrow

60 GPU Insights와 ProfileGPU 화면 · Tom Looman

61 New GPU Profiler and RHI Submission Pipeline · Epic

62 Graphics Programming Overview · Epic